



DEAKIN UNIVERSITY

Faculty of Science, Engineering and Built Environment

GPS Spoofing Detection for PX4-Based Unmanned Aerial Vehicles

A Software-Only Approach Using Sensor Fusion
and Machine Learning

Submitted as Research Report for
SIT746 Research Project (Advanced)

Submission Date: T1-2026

May 2026

Nikhil Sarwara

Student ID: 224234862
Master of Software Engineering (S464)

Supervised by: Dr. Ananda Maiti

Faculty of Science, Engineering and Built Environment
Deakin University

Abstract

Global Positioning System (GPS) spoofing poses a critical security threat to unmanned aerial vehicles (UAVs), as adversaries can transmit counterfeit signals to manipulate a drone's perceived position without triggering conventional failure modes. Existing countermeasures frequently depend on specialised hardware—multi-antenna arrays, military-grade encrypted signals, or high-grade inertial navigation systems—that are incompatible with the size, weight, and power constraints of commercial UAV platforms. This research presents a software-only GPS spoofing detection system for PX4-based drones that leverages cross-validation between GPS-reported position and inertial measurement unit (IMU) data to distinguish spoofing from environmental disturbances such as wind. The system employs a multi-layered detection pipeline combining eight physics-based heuristic rules with machine learning classifiers—Random Forest and one-dimensional Convolutional Neural Network—operating on 30-sample sliding windows of telemetry data acquired at 10 Hz via the MAVLink protocol. Evaluated on a dataset of 48,207 telemetry records from 8 simulated flight runs and 60 synthetic trajectories in a PX4 Software-In-The-Loop environment with Gazebo, the Random Forest achieved 98% accuracy and 0.99 normal-class F1 on 1,734 test windows, while the 1D CNN achieved 98% overall accuracy but failed to detect spoofed windows due to class imbalance reversal. An area-specific model trained on real Baylands flight data achieved 0.99 accuracy with 0.98 spoofed-class F1. The key contribution is a physics-based GPS–IMU consistency discriminator that distinguishes spoofing from environmental noise without requiring prior training on attack signatures. A graduated GPS Trust Index is proposed for real-time mitigation. The results demonstrate that real-time software-only spoofing detection is feasible, though the 3-second detection window exceeds theoretical safe-reaction times for high-speed scenarios, motivating future work on reduced-latency architectures and physical hardware validation.

Acknowledgements

I would like to express my sincere gratitude to my supervisors for their invaluable guidance, constructive feedback, and continuous support throughout this research project. Their expertise in cyber-physical systems and embedded security has been instrumental in shaping the direction and quality of this work.

I am grateful to Deakin University and the Faculty of Science, Engineering and Built Environment for providing the resources, laboratory facilities, and academic environment that made this research possible.

Special thanks to my colleagues and peers for their thoughtful discussions, code reviews, and encouragement during the development of this system.

Finally, I wish to thank my family and friends for their unwavering patience, understanding, and support throughout my research journey. Their encouragement has been a constant source of motivation.

List of Abbreviations

C/N ₀	Carrier-to-Noise Ratio
CNN	Convolutional Neural Network
DEM	Digital Elevation Model
DOI	Digital Object Identifier
EKF2	Extended Kalman Filter 2
EMA	Exponential Moving Average
ENU	East-North-Up (coordinate system)
FPR	False Positive Rate
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GTI	GPS Trust Index
HDOP	Horizontal Dilution of Precision
IMU	Inertial Measurement Unit
INS	Inertial Navigation System
MAVLink	Micro Air Vehicle Link (communication protocol)
MEMS	Micro-Electromechanical Systems
ML	Machine Learning
MSL	Mean Sea Level
OSNMA	Open Service Navigation Message Authentication
PID	Process Identifier
PVT	Position, Velocity, and Time
PX4	PX4 Autopilot (open-source flight stack)
RF	Random Forest
RQ	Research Question
RTL	Return-to-Launch
SDF	Simulation Description Format
SDR	Software-Defined Radio
SITL	Software-In-The-Loop
SWaP	Size, Weight, and Power
TUI	Terminal User Interface
UAV	Unmanned Aerial Vehicle
UDP	User Datagram Protocol
VDOP	Vertical Dilution of Precision

Contents

Acknowledgements	i
List of Abbreviations	iii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Research Aims and Objectives	3
1.5 Scope and Significance	4
1.6 Thesis Structure	4
2 Literature Review	6
2.1 GPS Fundamentals and Vulnerabilities	6
2.2 GPS Spoofing: Threats and Attack Vectors	7
2.3 Existing Detection Approaches	8
2.3.1 Cryptographic Authentication	8
2.3.2 Physical-Layer Techniques	8
2.3.3 Sensor-Fusion Methods	9
2.4 PX4 Architecture and the EKF2	9
2.5 Sensor Fusion for Anomaly Detection	10
2.5.1 EKF Innovation Monitoring and Rejection Latency	11
2.6 Machine Learning for Intrusion Detection	11
2.7 Research Gaps	13
3 System Design and Architecture	14
3.1 Design Requirements	14
3.1.1 Functional Requirements	14
3.1.2 Non-Functional Requirements	15
3.2 Overall System Architecture	15
3.3 Component Overview	16
3.4 Key Innovation: GPS–IMU Consistency Check	16
3.4.1 Physical Principle	17
3.4.2 Mathematical Formulation	17
3.4.3 Worked Numerical Example	18
4 Methodology	19
4.1 Research Methodology	19
4.2 Simulation Environment	19
4.2.1 Gazebo Wind and Physics Models	20

4.2.2	Docker Container Orchestration	21
4.3	Data Capture Pipeline	21
4.3.1	MAVLink Message Types and Telemetry Sources	21
4.3.2	Capture Procedure	22
4.4	Data Preprocessing	22
4.4.1	Stage 1: Cleaning	22
4.4.2	Stage 2: Feature Engineering	23
4.4.3	Stage 3: Heuristic Labelling	23
4.4.4	Threshold Sensitivity Analysis	25
4.4.5	Limitations of Heuristic Labelling	25
4.4.6	Additional Evaluation Metrics	25
4.4.7	Stage 4: Sliding Windows	26
4.4.8	Stage 5: Dataset Splitting	26
4.5	Machine Learning Model Architecture	27
4.5.1	Random Forest	27
4.5.2	1D Convolutional Neural Network	27
4.6	Evaluation Metrics	28
5	Implementation	29
5.1	Telemetry Acquisition Module	29
5.1.1	MAVLinkClient Architecture	29
5.1.2	State Model	30
5.1.3	CSV Logging Format	30
5.1.4	Message Rate Analysis	30
5.1.5	Message Rate Analysis	31
5.2	Data Preprocessing	32
5.2.1	Healing and Imputation Strategy	32
5.2.2	Time Alignment Between Disparate Message Rates	32
5.3	Auto-Labelling System	33
5.3.1	Heuristic Rule Tuning and Threshold Calibration	33
5.3.2	Label Aggregation and Conflict Resolution	34
5.4	Feature Engineering and Windowing	34
5.5	ML Model Training	34
5.5.1	Random Forest Training	35
5.5.2	1D CNN Training	35
5.5.3	Training Pipeline Modes	35
5.5.4	Pipeline Configuration Management	36
5.5.5	Cross-Validation Strategy for Small Datasets	36
5.5.6	Model Serialization and Deployment Versioning	36
5.6	Real-Time Inference Engine	37
5.6.1	Terrain-Aware Model Dispatching	37
5.7	Management Interfaces	38
5.7.1	TUI Management Console	38
5.7.2	Streamlit Web Dashboard	39
5.7.3	Process Manager and Component Lifecycle	41
5.8	GPS Spoofing Attack Injector	41

6	Results and Evaluation	44
6.1	Dataset Characterisation	44
6.1.1	Environmental Conditions and Model Impact	45
6.2	Model Performance and Comparative Analysis	46
6.2.1	Random Forest Classification	46
6.2.2	1D CNN Classification	47
6.2.3	Comparative Analysis	48
6.2.4	CNN Training Dynamics and Overfitting Analysis	49
6.2.5	Random Forest Feature Importance	50
6.2.6	Ablation Study	50
6.3	Live Inference Results	51
6.4	Terrain-Aware Detection	52
6.5	Latency Analysis	53
7	Discussion	55
7.1	Interpretation of Key Findings	55
7.2	Comparison with Existing Work	56
7.3	Limitations	57
7.4	Threats to Validity	59
7.5	Proposed GPS Trust Index	59
7.5.1	Mathematical Design	60
7.5.2	Graduated Response Protocol	60
8	Conclusion and Future Work	63
8.1	Summary of Contributions	63
8.2	Answers to Research Questions	64
8.3	Practical Implications	65
8.4	Future Work	66
8.5	Concluding Remarks	67
	Bibliography	68
A	Supplementary Material	70
A.1	Project Directory Structure	70
A.2	Configuration File	71
A.3	Commands Reference	71
A.4	Ethics and Compliance	72

List of Figures

2.1	GPS spoofing attack flow: the attacker transmits a counterfeit signal structurally compliant with IS-GPS-200, causing the victim PX4 receiver to accept falsified PVT estimates while reporting a healthy fix.	7
2.2	PX4 EKF2 sensor fusion architecture. GPS, IMU, barometer, and magnetometer data feed into the 24-state Extended Kalman Filter. Under spoofing, the GPS input (highlighted in red) reports low error metrics, causing the EKF2 to weight it heavily despite containing falsified coordinates.	10
3.1	System architecture showing data flow from PX4 simulation through GPS Monitor, ML Pipeline, and Live Inference to Dashboard. The attack injector provides controlled spoofing for testing.	15
3.2	GPS–IMU consistency principle. Left: under a wind gust, both GPS and IMU detect the physical displacement—they agree. Right: under GPS spoofing, GPS reports position change while IMU registers no motion—the cross-sensor inconsistency reveals the attack.	17
4.1	PX4 SITL simulation running in Gazebo Harmonic with the Iris x500 quadcopter model in the windy world.	20
4.2	QGroundControl mission planning interface showing the drone waypoint path on the map with telemetry instrument panel during a scripted flight.	22
4.3	Heuristic labelling decision flow. Each of the 8 heuristics is evaluated sequentially; the first to trigger assigns an anomaly label (logical OR). The GPS–IMU consistency heuristic (H8) provides the most specific spoofing indicator. Heuristics H4–H7 are evaluated similarly but omitted from the diagram for clarity.	24
4.4	Sliding window construction with window length $n = 30$ (3 seconds at 10 Hz) and stride $s = 15$ (50% overlap). Red dots represent spoofed telemetry samples; an attack onset at sample 8 is captured by both overlapping windows W_t and W_{t+15}	26
5.1	GPS Monitor terminal display showing real-time GPS and IMU telemetry captured from PX4 SITL at 10 Hz.	31

5.2	TUI Management Console main dashboard showing component controls, status table, data file explorer, ML pipeline buttons, terrain model status, and GPS spoofer panel with six attack mode buttons. The spoofer console shows a completed drift attack with 293 GPS_INPUT frames injected at 3 m/s.	39
5.3	Streamlit web dashboard showing the live monitor tab with GPS Trust Index gauge (0–1 scale), current telemetry values, and anomaly probability line chart during normal flight operation.	40
5.4	Streamlit dashboard during a GPS spoofing attack event. The anomaly probability has exceeded the 0.3 detection threshold, displaying a red ANOMALY status indicator and a degraded GPS Trust Index.	40
5.5	Streamlit dashboard log replay tab showing an uploaded inference CSV log displayed as a scrollable data table with anomaly probability, GPS coordinates, and terrain context for each detection window.	41
5.6	Port architecture for simultaneous operation of all components across three independent MAVLink instances.	42
5.7	GPS spoofer terminal output during a drift attack at 3 m/s intensity. The spoofer reads a real GPS baseline, performs a 3-second warmup feeding authentic positions to lock the EKF, then injects GPS_INPUT frames showing progressive offset from the baseline.	43
5.8	ML Live Inference engine running on streaming MAVLink telemetry, showing terrain-aware detection output with terrain classification (flat), model type (rf), anomaly probability, and NORMAL/ANOMALY status for each processed window.	43
6.1	Random Forest validation confusion matrix showing high normal-class recall with a low false positive rate on the 1,701 normal validation windows.	47
6.2	1D CNN confusion matrix showing perfect normal-class recall but complete failure to detect spoofed windows (0 true positives).	48
6.3	F1-score comparison between the RF and CNN on the test set, highlighting the RF's 63% spoofed-class F1 versus the CNN's complete failure to detect spoofed windows.	49
6.4	1D CNN training and validation loss curves over 50 epochs. Validation loss plateaus early, and overfitting becomes evident after approximately epoch 25. Early stopping activated at epoch 38.	49
6.5	Random Forest top-20 feature importance scores ranked by Gini impurity decrease. Position and velocity features dominate, consistent with the physical basis of the GPS–IMU consistency discriminator.	50
6.6	Gazebo simulation world selector in the TUI, showing available worlds with corresponding terrain classifications (flat, mountain, sea) for terrain-aware model dispatching.	53

6.7	Detection latency timeline. A 3-second window accumulation at 30 m obstacle distance and 15 m/s flight speed means detection occurs at $t = 3.2\text{ s}$ but the available reaction window closes at $t = 2.0\text{ s}$	54
7.1	GPS Trust Index graduated response protocol. As the GTI declines, protective measures escalate from sensor weight modulation through hold mode to emergency return-to-launch, with MAVLink alerts broadcast at each escalation step.	61

List of Tables

2.1	Comparison of ML-based GNSS spoofing detection approaches.	12
4.1	MAVLink telemetry sources, frequencies, and detection roles.	21
4.2	Feature set organised by category.	23
4.3	Heuristic labelling rules with thresholds.	23
4.4	1D CNN architecture.	27
6.1	Dataset summary statistics after full pipeline processing.	44
6.2	Per-flight experimental breakdown for the 8 simulated flights.	45
6.3	Random Forest classification performance on the test set (1,734 windows). . . .	46
6.4	1D CNN classification performance on the test set (1,734 windows).	47
6.5	Model performance comparison on the combined test set.	48
6.6	CNN ablation study results on the test set.	51
6.7	Live inference log summary for a representative flight session.	52
7.1	Structured comparison of GPS spoofing detection approaches.	57

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of unmanned aerial vehicles (UAVs) across diverse operational domains—including civilian infrastructure inspection, precision agriculture, emergency response, environmental monitoring, and military reconnaissance—has elevated concerns regarding their vulnerability to Global Positioning System (GPS) spoofing attacks [7]. In such attacks, adversaries transmit counterfeit GPS signals synthesised with correct timing and structure but deliberately manipulated coordinates to usurp legitimate satellite navigation. The economic footprint of the commercial drone market is projected to reach \$54.6 billion by 2030, making the security of these platforms a matter of significant practical concern.

Unlike jamming, which merely attenuates or disrupts the GPS signal causing loss of lock, spoofing deceives the receiver into accepting falsified position, velocity, and time (PVT) estimates without any observable degradation in signal quality. Jamming denies service; spoofing corrupts service. A jamming attack is readily detected through carrier-to-noise ratio degradation. A spoofing attack, by contrast, is silent: the receiver continues to report a healthy fix while computing an incorrect but internally consistent PVT solution. The PX4 flight stack—which powers a substantial fraction of commercial and research drones worldwide—relies heavily on GPS as a primary input to its Extended Kalman Filter (EKF2) for state estimation [21]. A successful spoofing attack can therefore cascade through the entire control pipeline, producing catastrophic trajectory deviations without triggering conventional failure modes [8].

The criticality of this threat was highlighted by a 2012 study in which researchers at the University of Texas successfully achieved full 3D control of a civilian UAV through GPS spoofing alone. In 2019, Iran claimed to have captured a US military drone by spoofing its GPS navigation. These incidents demonstrate that GPS spoofing is not a theoretical vulnerability but a practical, exploitable attack vector. Modern software-defined radios (SDRs) capable of generating counterfeit GPS signals are available for under \$300, dramatically lowering the barrier to entry for potential adversaries.

Existing countermeasures frequently depend on specialised hardware—multi-antenna arrays for

angle-of-arrival discrimination, high-grade inertial navigation systems (INS) with tightly coupled integration, or military-grade encrypted signals (M-code, SAASM)—that are incompatible with the SWaP-constrained (Size, Weight, and Power) platforms that dominate the commercial UAV market. Cryptographic authentication schemes such as Galileo’s Open Service Navigation Message Authentication (OSNMA) are promising but remain unavailable on the vast installed base of civilian receivers. A software-only detection mechanism that leverages sensors already present on standard PX4 autopilots represents a compelling and cost-effective alternative, provided it can achieve sufficient discriminative power in real time.

1.2 Problem Statement

The PX4 autopilot employs EKF2, a 24-state Extended Kalman Filter that fuses GPS, barometer, magnetometer, and IMU data to estimate position, velocity, attitude, and sensor biases. Under normal operation, EKF2 weights GPS position updates according to their reported accuracy (horizontal position error eph , vertical position error epv). When spoofed GPS data is injected, the EKF2 accepts the falsified position as a high-confidence observation because the spoofed signal reports low horizontal dilution of precision (HDOP) and healthy accuracy metrics.

The filter state is consequently dragged toward the attacker-controlled coordinates, while the IMU integration—which continues to reflect the drone’s true physical motion—accumulates increasing innovation residuals relative to the corrupted GPS input. The EKF2’s internal innovation monitoring will eventually reject GPS observations when the residual exceeds its configurable outlier gate. However, the latency of this natural rejection mechanism is unacceptably long for safety-critical flight operations. A dedicated anomaly detector that actively cross-validates GPS against IMU-derived motion estimates can flag spoofing before the EKF2 state is significantly corrupted.

The core research challenge is therefore: can we construct a practical, real-time GPS spoofing detector for commercial PX4 drones using only the sensors and telemetry already available, without requiring hardware modifications or firmware changes?

To make this vulnerability concrete, consider a simplified numerical scenario. A drone hovers at a true position of $(0, 0, 20)$ m in the local ENU frame. At $t = 0$, a spoofing attack begins, injecting GPS positions that drift eastward at 2 m/s. At $t = 1$ s, the spoofed GPS reports $(0, 2, 20)$. The true drone position, as measured by the IMU, remains $(0, 0, 20)$ —it has not physically moved. The EKF2 innovation (the difference between predicted position $(0, 0, 20)$ and observed position $(0, 2, 20)$) is 2 m. With a typical PX4 innovation gate of 5–10 m (configurable via the `EKF2_GPS_P_GATE` parameter), the spoofed observation is accepted as valid. After 5 seconds, the drone’s EKF2 state has been corrupted by 10 m of eastward error, and the innovation has grown to 10 m, potentially exceeding the gate. The drone has been flying on corrupted position estimates for 5 seconds—in that time, at 15 m/s, it would have travelled 75 m toward a potential obstacle. This numerical illustration demonstrates why a dedicated, rapid-responding spoofing detector is essential: the EKF2’s natural protection mechanism is too slow for safety-critical

flight.

1.3 Research Questions

This study is organised around three interdependent research questions:

RQ1: What are the primary vulnerabilities of GPS in autonomous mobile robots to spoofing attacks? This question addresses the architectural and systemic weaknesses that enable spoofing to succeed, with specific focus on the PX4 flight stack.

RQ2: How can sensor fusion techniques effectively detect GPS spoofing in real time? This question operationalises the core insight that spoofing introduces a cross-sensor inconsistency between GPS-reported position and IMU-detected physical motion.

RQ3: How can a dynamic GPS Trust Index be designed to quantify the reliability of GPS data? This question extends detection from binary classification to continuous confidence metrics suitable for safety-critical operational decision-making.

To evaluate RQ2 and RQ3, the following hypotheses guide the empirical analysis:

- H1: GPS spoofing introduces measurable inconsistencies between GPS-reported position and inertial measurement unit (IMU) data that are distinguishable from normal flight disturbances.
- H2: Physics-based heuristic rules can generate sufficiently accurate training labels without requiring laborious manual annotation of flight data.
- H3: Machine learning classifiers—specifically Random Forest and one-dimensional Convolutional Neural Networks—can distinguish normal flight windows from spoofed segments using only standard PX4 telemetry.
- H4: A live inference pipeline can process streaming MAVLink telemetry with latency suitable for real-time onboard detection.

1.4 Research Aims and Objectives

The primary objectives of this research are:

1. **Develop** a real-time GPS spoofing detection system for PX4-based drones using software-only techniques deployable on existing hardware without firmware modification.
2. **Create** an automated labelling mechanism using eight physics-based heuristic rules that eliminates the need for manually annotated training data.
3. **Train and evaluate** two complementary machine learning models—Random Forest and 1D Convolutional Neural Network—for accurate binary classification of GPS behaviour.

4. **Design and implement** a comprehensive management interface integrating terminal-based component control with web-based monitoring and log visualisation.
5. **Validate** the complete system through controlled simulated attack scenarios in a reproducible PX4 Software-In-The-Loop environment.
6. **Propose** a graduated GPS Trust Index and post-detection response protocol for operational use.

1.5 Scope and Significance

This research is scoped to PX4-based quadcopter platforms operating in the PX4 Software-In-The-Loop (SITL) simulation environment with Gazebo physics engine. The detection approach is exclusively software-based, leveraging standard telemetry data available on commercial PX4 autopilots without requiring hardware augmentation, firmware customisation, or access to encrypted GPS signals. The experimental scope covers 8 simulated flight runs with scripted rectangular survey patterns and controlled GPS spoofing injection via MAVLink GPS_INPUT messages.

The significance of this work lies in demonstrating that cost-effective, real-time GPS spoofing detection is achievable using commodity sensor hardware and open-source software. By bridging the gap between theoretical academic demonstrations of GPS vulnerability and practical deployable countermeasures, this research contributes to the growing body of work on UAV cyber-physical security. The modular, open-source nature of the implementation enables reproducibility, extension, and adaptation to other autonomous platforms beyond the specific PX4 ecosystem.

To clarify the boundaries of this investigation, the following are explicitly out of scope: physical hardware flight tests (all validation is simulation-based), multi-drone swarm coordination scenarios, encrypted military GPS signal analysis, real-world radio-frequency spoofing from software-defined radio hardware, non-quadcopter platforms (fixed-wing, VTOL), and non-PX4 flight stacks (ArduPilot, BetaFlight). While these exclusions limit the immediate generalisability of findings, they define a focused, achievable scope appropriate for the resources and timeline of this research project.

1.6 Thesis Structure

The remainder of this thesis is organised as follows. **Chapter 2** reviews the relevant literature on GPS fundamentals, spoofing attack mechanisms, existing detection approaches from cryptographic, physical-layer, and sensor-fusion paradigms, PX4 architecture, and machine learning for anomaly detection. **Chapter 3** presents the high-level system design and architecture, including the key innovation of the GPS-IMU consistency discriminator. **Chapter 4** details the research methodology, simulation environment, data capture pipeline, preprocessing

stages, heuristic labelling rules, model architectures, and evaluation metrics. **Chapter 5** describes the implementation of each system component with architectural detail and code-level documentation. **Chapter 6** presents the experimental results including dataset characterisation, model performance, the CNN Paradox analysis, live inference results, terrain-aware detection, and latency analysis. **Chapter 7** provides interpretation of findings, comparison with existing work, discussion of limitations and threats to validity, and the proposed GPS Trust Index with graduated response protocol. **Chapter 8** concludes the thesis with a summary of contributions, formal answers to each research question, practical implications, and directions for future work. Supplementary material is provided in the Appendix.

Chapter 2

Literature Review

This chapter surveys the existing scholarly work relevant to GPS spoofing detection in unmanned aerial vehicles. The review is organised around six thematic areas: GPS fundamentals and vulnerabilities, spoofing attack mechanisms, existing detection approaches, PX4 flight stack architecture, sensor fusion techniques, and machine learning for anomaly detection. The chapter concludes by identifying specific research gaps that this work addresses.

2.1 GPS Fundamentals and Vulnerabilities

The Global Positioning System, a satellite-based radio-navigation system owned by the United States government and operated by the United States Space Force, provides geolocation and time information to GPS receivers anywhere on or near Earth [23]. The system architecture comprises three segments: the space segment, consisting of a nominal constellation of 31 satellites broadcasting on the L1 frequency (1575.42 MHz) using Coarse/Acquisition (C/A) code and on the L2 frequency (1227.60 MHz); the control segment, comprising a global network of ground stations that monitor satellite health, ephemeris, and clock corrections; and the user segment, consisting of commercial GPS receivers that compute PVT solutions from time-of-flight measurements to at least four satellites.

Civilian GPS employs the C/A code on L1, which is publicly documented in the GPS Interface Specification IS-GPS-200. While this openness has enabled widespread adoption across transportation, agriculture, telecommunications, and defence sectors, it creates inherent security vulnerabilities [22]. The signal structure is fully known, the navigation message is unauthenticated, and civilian receivers possess no mechanism to cryptographically verify the origin of received signals. This contrasts sharply with military GPS, which utilises encrypted Precision (P/Y) code and anti-spoofing modules that are inaccessible to civilian users.

GPS signals at the Earth's surface are extremely weak—approximately -130 dBm, or below the thermal noise floor—making them susceptible to both intentional and unintentional interference. The low signal power enables a fundamental vulnerability: an adversary can generate a stronger, structurally valid counterfeit signal that overpowers the genuine satellite transmission, causing the

receiver to track the spurious signal instead.

2.2 GPS Spoofing: Threats and Attack Vectors

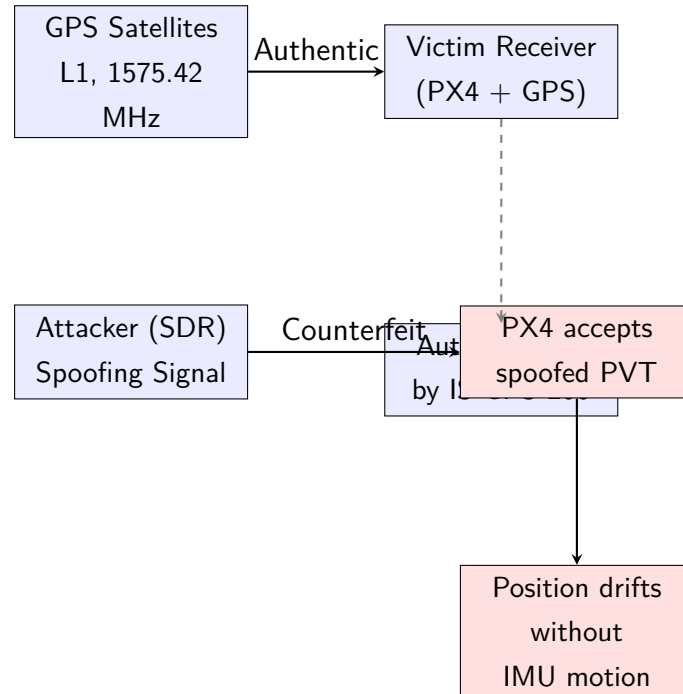


Figure 2.1: GPS spoofing attack flow: the attacker transmits a counterfeit signal structurally compliant with IS-GPS-200, causing the victim PX4 receiver to accept falsified PVT estimates while reporting a healthy fix.

The cybersecurity literature often conflates two distinct categories of GPS interference: jamming and spoofing. Understanding their operational differences is critical for designing effective countermeasures.

Jamming broadcasts high-power noise in the GPS frequency band, elevating the noise floor until the receiver loses carrier-phase lock. Detection of jamming is straightforward: the receiver reports a precipitous drop in carrier-to-noise ratio (C/N_0) across all tracked satellites, followed by an explicit loss-of-fix status. Jamming is a denial-of-service attack; the receiver knows it has lost GPS.

Spoofing, in contrast, transmits a carefully constructed replica of authentic GPS signals with manipulated navigation data. Because the counterfeit signals comply with the IS-GPS-200 signal-in-space specification, the victim receiver synchronises to the attacker-generated signals, computes an incorrect but internally consistent PVT solution, and reports a healthy fix with normal C/N_0 and accuracy metrics [26]. Spoofing is a corruption-of-service attack; the receiver believes it has accurate GPS.

Kerns et al. [8] provided the first comprehensive open-source demonstration of civilian GPS spoofing using a low-cost software-defined radio platform, establishing that the threat is neither theoretical nor prohibitively expensive. Their work showed that a \$1,000 SDR could generate GPS

signals that successfully captured a variety of commercial receivers. Humphreys [7] subsequently demonstrated full 3D control of a civilian unmanned aerial vehicle through GPS spoofing, raising significant policy and security concerns.

More recent research has explored increasingly sophisticated attack variants [17]:

- **Slow-drift spoofing:** Gradually introducing position error at rates below typical detection thresholds (less than 0.5 m/s), causing the EKF to slowly converge to the falsified position without triggering innovation monitoring.
- **Meaconing:** Recording and delayed rebroadcast of authentic GPS signals, which preserves the spatial consistency of the satellite geometry but introduces controlled temporal and positional offsets.
- **Multi-vehicle coordinated spoofing:** Simultaneously attacking multiple UAVs in a swarm with coordinated false positions that maintain inter-vehicle geometric consistency.
- **Carry-off attacks:** Starting with signals perfectly aligned to the authentic GPS and then smoothly increasing the divergence, making it difficult for the receiver to detect the transition.

2.3 Existing Detection Approaches

GPS spoofing detection has been approached through three broad paradigms: cryptographic authentication, physical-layer signal analysis, and sensor-fusion cross-validation.

2.3.1 Cryptographic Authentication

Cryptographic methods embed digital signatures or encryption in the GPS navigation message, enabling receivers to verify signal authenticity [16]. Notable implementations include Galileo's Open Service Navigation Message Authentication (OSNMA), GPS Chimera, and the military M-code with Selective Availability Anti-Spoofing Module (SAASM). While cryptographically robust, these approaches require modifications to the satellite constellation, ground infrastructure, and receiver firmware. They are unavailable to the vast installed base of civilian receivers and cannot be retrofitted to existing PX4 flight controllers. Furthermore, cryptographic authentication cannot detect meaconing attacks where the signal itself is authentic but its timing has been manipulated.

2.3.2 Physical-Layer Techniques

Physical-layer methods exploit signal characteristics to discriminate authentic from counterfeit transmissions without cryptographic verification [11]:

- **Angle-of-Arrival (AOA) discrimination:** Comparing signal direction estimates across multiple antennas to detect spatially inconsistent signals from a single spoofing source.

- **Automatic Gain Control (AGC) monitoring:** Detecting the power increase associated with the introduction of a spoofing signal by monitoring receiver AGC levels.
- **Correlator output analysis:** Examining the distortion of the correlator peak shape that occurs when counterfeit and authentic signals overlap.
- **Vestigial signal detection:** Identifying residual authentic signals that remain visible below the stronger counterfeit signal.

These approaches require hardware augmentation—additional RF front-end components, multiple antennas, or modified receiver firmware—that is generally impractical for SWaP-constrained commercial UAV platforms.

2.3.3 Sensor-Fusion Methods

Sensor-fusion approaches leverage the physical inconsistency between GPS-reported motion and measurements from other onboard sensors [2]. The fundamental insight is that GPS spoofing affects only the position measurement domain, while inertial and environmental sensors remain faithful to true physical motion. This cross-sensor inconsistency is independent of the attacker's signal generation technique and cannot be concealed by improving the spoofing waveform fidelity.

Broumandan et al. [2] demonstrated that IMU-GPS inconsistency provides a robust spoofing indicator, particularly when integrated with INS-grade accelerometers and gyroscopes. Guizzaro et al. [5] extended this work by applying neural network classifiers to IMU measurements for real-time spoofing detection. Manesh et al. [14] developed a detection system specifically for UAV platforms using a combination of GPS signal metrics and machine learning.

The present work differs from these prior contributions in several important respects. First, we target standard low-cost PX4 hardware telemetry rather than navigation-grade IMUs. Second, we implement the cross-sensor check as both a standalone heuristic and an input feature to learned classifiers. Third, we integrate detection into a complete end-to-end pipeline with automated labelling, terrain-aware model selection, and graduated response protocols.

2.4 PX4 Architecture and the EKF2

The PX4 autopilot is an open-source flight control software stack widely adopted in both research and commercial UAV applications [21]. Its state estimation system is built around EKF2, a 24-state Extended Kalman Filter that performs real-time sensor fusion with the following state vector components: position (3), velocity (3), attitude quaternion (4), delta-angle bias (3), delta-velocity bias (3), wind velocity (2), geomagnetic field (3), and magnetic field bias (3).

Under normal operation, EKF2 fuses observations from multiple sensor sources: GPS for absolute position and velocity, barometer for altitude, magnetometer for heading, and IMU for high-rate inertial propagation. Each observation is weighted according to its reported accuracy metrics. GPS observations are weighted by eph (horizontal position error) and epv (vertical

position error). The innovation—the difference between the predicted measurement and the actual observation—is compared against a configurable threshold gate; observations exceeding this gate are rejected as outliers.

This architecture creates a specific vulnerability to GPS spoofing [27]: because the spoofed signal artificially reports low eph/epv values, the EKF2 accepts the falsified position as a high-confidence observation. The filter state is progressively dragged toward the attacker-controlled coordinates, while IMU integration accumulates increasing innovation residuals. The natural EKF rejection mechanism triggers only when innovations have grown sufficiently large, which may require tens of seconds depending on the spoofing intensity and the specific gate configuration. During this period, the drone’s control loops are acting on corrupted position estimates [18].

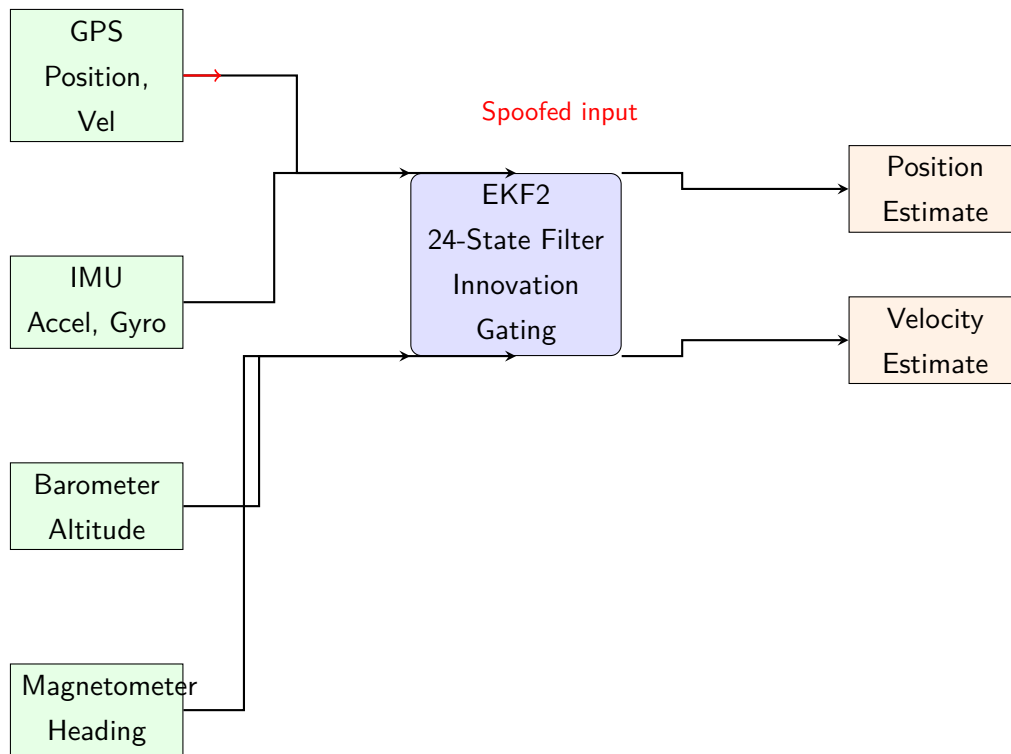


Figure 2.2: PX4 EKF2 sensor fusion architecture. GPS, IMU, barometer, and magnetometer data feed into the 24-state Extended Kalman Filter. Under spoofing, the GPS input (highlighted in red) reports low error metrics, causing the EKF2 to weight it heavily despite containing falsified coordinates.

2.5 Sensor Fusion for Anomaly Detection

The theoretical foundation for the GPS–IMU consistency check is rooted in classical inertial navigation mechanics. An IMU contains three orthogonal accelerometers that measure specific force (acceleration plus gravitational component) and three orthogonal gyroscopes that measure angular rate [10]. The specific force vector, when resolved in the navigation frame, corrected for gravity, and double-integrated, yields position displacement. GPS provides absolute position measurements at lower update rates but with bounded long-term error.

Under normal flight dynamics, the displacement estimated from IMU integration and the displacement computed from GPS positions should agree within bounds determined by IMU noise characteristics, gravity modelling errors, and integration drift [13]. Wind gusts affect both sensor domains: GPS position changes because the vehicle is physically displaced, and IMU registers the corresponding acceleration and angular rates. GPS spoofing, however, introduces an asymmetry: GPS position changes without corresponding IMU-detected physical motion. This creates a measurable discrepancy in the cross-sensor consistency that is independent of the attacker's signal generation method.

The discriminative power of this inconsistency derives from fundamental physics rather than from assumptions about attack characteristics [1]. No spoofing technique—regardless of waveform fidelity, power level, or transition smoothness—can prevent the GPS-reported motion from diverging from the IMU-observed physical motion, because the attacker cannot physically manipulate the drone's inertia.

2.5.1 EKF Innovation Monitoring and Rejection Latency

The PX4 EKF2 includes built-in innovation monitoring that provides a natural, albeit slow, spoofing detection mechanism. Key diagnostic parameters streamed in the ESTIMATOR_STATUS MAVLink message include: `vel_innov` (velocity innovation), `pos_horiz_innov` (horizontal position innovation), and `pos_vert_innov` (vertical position innovation), each representing the Mahalanobis distance between the EKF-predicted measurement and the actual sensor observation. When an innovation value exceeds the configurable gate `EKF2_GPS_P_GATE` (default: 5.0), the corresponding GPS observation is rejected, and the EKF propagates using IMU dead reckoning alone.

Under a typical spoofing attack with 2 m/s position drift, the horizontal position innovation builds up at approximately 2 m per second. With a default gate of 5.0, the innovation will exceed the gate after approximately 2.5 seconds. During this period, the EKF state estimate has already been corrupted by 5 m of position error. The `GPS_1_PROTOCOL = 1` (MAVLink) parameter, required for the attack injector to function, further bypasses the receiver-level signal quality checks that would normally provide early warning. This EKF-level vulnerability directly motivates the companion detection system developed in this research, which aims to detect spoofing before the EKF innovation has grown large enough to trigger natural rejection.

2.6 Machine Learning for Intrusion Detection

The application of machine learning to GPS anomaly detection has gained significant attention in recent years. Two complementary model families have proven particularly effective for this domain.

Random Forest classifiers offer several advantages for GPS spoofing detection [12]: they handle mixed feature types (continuous, categorical, binary) natively without extensive feature

engineering; they are inherently robust to class imbalance through bootstrap aggregation and class weighting; they provide feature importance metrics that aid interpretability and model diagnostics; and they require minimal hyperparameter tuning compared to deep learning approaches. Random Forests have been successfully applied to GNSS spoofing detection by Guizzaro et al. [5] and to broader UAV security tasks by Khan et al. [9].

One-dimensional Convolutional Neural Networks (1D CNNs) are well-suited to multivariate time-series classification tasks such as GPS telemetry analysis [25]. By applying convolutional filters along the temporal dimension, 1D CNNs can learn discriminative temporal patterns—such as the characteristic onset transient of a spoofing attack—without requiring manual feature extraction. Deep learning approaches for GNSS anomaly detection have been explored by Dang et al. [4] for cellular-connected UAV swarms and by Chen et al. [3] for autonomous drone navigation.

Key challenges confronting ML-based GPS spoofing detection include severe class imbalance (normal flight dominates operational data by orders of magnitude), temporal dependency (consecutive telemetry samples are not independent observations), concept drift (the characteristics of normal flight evolve with the operational environment), and the need for real-time inference on resource-constrained embedded platforms. The sliding window approach adopted in this thesis addresses temporal dependency; the automated heuristic labelling pipeline addresses the annotation bottleneck; and the terrain-aware model dispatching addresses environmental concept drift.

Table 2.1 summarises prior machine learning approaches to GNSS spoofing detection, providing a structured comparison with the present work.

Table 2.1: Comparison of ML-based GNSS spoofing detection approaches.

Study	Sensors	Model	Real-Time	PX4 Compat.
Manesh et al. [14]	GPS metrics only	MLP	No (offline)	No
Guizzaro et al. [5]	IMU + GPS	Neural Net	Yes	No
Dang et al. [4]	GPS, cellular	GNN	Yes	No
Chen et al. [3]	GPS + vision	Deep NN	Yes	No
This work	IMU + GPS + EKF	RF + CNN	Yes	Yes

The present work is distinguished from prior approaches by three characteristics: (a) explicit targeting of the PX4 ecosystem with compatibility verification, (b) use of EKF innovation metrics as features alongside raw sensor data, providing both physical and filter-level anomaly signals, and (c) a cascaded architecture where physics-based heuristics and learned classifiers complement each other, enabling detection before training data is available while improving discrimination as labelled data accumulates.

2.7 Research Gaps

A systematic survey of the literature reveals several specific gaps that this research addresses:

1. **No existing software-only detector has been rigorously evaluated on PX4 platforms.** While prior work has demonstrated GPS spoofing detection using IMU cross-validation, these demonstrations either assumed navigation-grade sensors, required hardware modifications, or were evaluated in post-hoc offline analysis rather than real-time streaming inference.
2. **The GPS–IMU consistency check has not been developed as a standalone, physics-based discriminator in a cascaded detection architecture.** Most sensor-fusion approaches treat IMU data as an additional feature channel for a learned classifier rather than leveraging it as a principled, model-free anomaly indicator.
3. **The latency requirements for safety-critical UAV GPS spoofing detection have not been formally analysed.** While real-time detection is frequently cited as a requirement, no prior work has quantified the relationship between detection window length, inference latency, and available reaction time for obstacle avoidance.
4. **No prior work has proposed a continuous GPS trust metric for graduated response.** Existing approaches typically produce binary detection outputs (spoofed / not spoofed), which are inadequate for safety-critical systems where the appropriate response may depend on the confidence of detection and the operational context.
5. **Most detection systems require manually labelled training data, which is expensive and impractical to produce for the diverse range of attack types and environmental conditions.** An automated labelling pipeline using physics-based heuristics would significantly reduce the barrier to deployment.

This research directly addresses these gaps by developing, implementing, and evaluating a complete GPS spoofing detection pipeline for PX4 UAVs that: (a) operates on standard telemetry without hardware augmentation; (b) employs a principled GPS–IMU consistency check as the core discriminator; (c) quantifies detection latency relative to safety constraints; (d) proposes a graduated GPS Trust Index for operational response; and (e) uses automated heuristic labelling to eliminate manual annotation requirements.

Chapter 3

System Design and Architecture

This chapter presents the high-level design and architecture of the GPS spoofing detection system. We discuss design requirements, the modular system architecture, individual component roles, and the key technical innovation—the GPS–IMU consistency discriminator.

3.1 Design Requirements

The system was designed to satisfy a set of functional and non-functional requirements derived from the research objectives and practical deployment constraints of the PX4 ecosystem.

3.1.1 Functional Requirements

1. **F1—Telemetry Acquisition:** Collect GPS, IMU, and vehicle state data from a PX4 flight controller at a minimum rate of 10 Hz via the MAVLink protocol [15].
2. **F2—Automated Labelling:** Automatically assign binary anomaly labels to each telemetry row using physics-based heuristic rules, eliminating the need for manual annotation.
3. **F3—Data Processing Pipeline:** Transform raw CSV telemetry logs through a five-stage pipeline: cleaning, feature engineering, labelling, windowing, and dataset splitting.
4. **F4—Model Training:** Train and serialise Random Forest and 1D Convolutional Neural Network classifiers from the labelled dataset [20, 19].
5. **F5—Real-Time Inference:** Perform streaming anomaly detection on live MAVLink telemetry with sub-second latency and log results for review.
6. **F6—Management Interface:** Provide both terminal-based component lifecycle management and web-based monitoring dashboard [24].
7. **F7—Attack Simulation:** Generate controlled GPS spoofing attacks in a PX4 SITL environment for system testing and validation.

3.1.2 Non-Functional Requirements

- **Inference latency:** Less than 100 ms for single-window prediction.
- **Detection accuracy:** Target above 85% on balanced test sets.
- **Sampling consistency:** Maintain stable 10 Hz logging without dropped messages during normal and spoofed flight segments.
- **Software-only deployment:** No hardware augmentation or firmware modifications required.
- **Modularity:** Each component (monitor, pipeline, inference, UI) runs independently and can be started/stopped without affecting others.

3.2 Overall System Architecture

The system follows a modular, pipeline-oriented architecture decomposed into eight logical subsystems following the “divide and conquer” decomposition principle. Figure 3.1 illustrates the overall architecture and data flow.

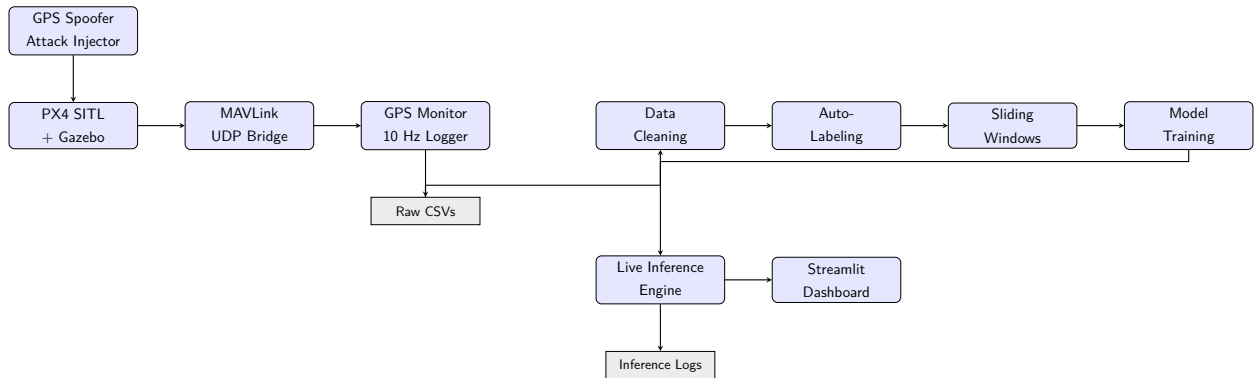


Figure 3.1: System architecture showing data flow from PX4 simulation through GPS Monitor, ML Pipeline, and Live Inference to Dashboard. The attack injector provides controlled spoofing for testing.

The data flow proceeds as follows:

1. PX4 SITL generates simulated flight physics and transmits telemetry via MAVLink over UDP port 14560.
2. The GPS Monitor connects as a MAVLink client, captures 8 message types at 10 Hz, and writes structured CSV logs.
3. The ML Pipeline reads raw CSV files and sequentially executes data cleaning, automated labelling, sliding window creation, and model training.
4. Trained models (RF and CNN) are serialised to disk for use by the inference engine.

5. The Live Inference Engine consumes streaming MAVLink telemetry, accumulates 30-sample windows, and outputs anomaly probabilities with terrain-aware model selection.
6. The Streamlit Dashboard displays live anomaly probabilities and a GPS Trust Index gauge, with historical log replay capability.

3.3 Component Overview

The system comprises five major runtime components and one testing utility:

GPS Monitor pymavlink-based telemetry collector. Maintains a thread-safe state model of GPS, IMU, Estimator, and Vehicle Health data. Logs structured CSV at 10 Hz with 34 fields per record. Includes a terminal-based real-time display with telemetry preview.

ML Pipeline Python pipeline orchestrator supporting four processing modes: `process-single` (single flight), `process-batch` (all flights in directory), `train` (model training only), and `full` (end-to-end pipeline). Configuration-driven via YAML for reproducibility.

Live Inference Engine Streaming detector with deque-buffered 30-sample sliding windows. Loads serialised models via joblib. Applies EMA smoothing for output stability. Supports terrain-aware model dispatching. Outputs labelled CSV log with per-window anomaly probability and metadata.

Streamlit Dashboard Web interface with two operational modes: live monitor (GPS Trust Index gauge, telemetry display, anomaly probability chart) and log replay (upload and visualise historical inference logs). Auto-refreshes from the live log file.

TUI Management Console Textual framework application providing component lifecycle management (start/stop with PID tracking), data file browser, ML pipeline execution buttons, integrated log viewer with colour-coded output, and one-click GPS spoofing attack launch.

GPS Spoofer MAVLink GPS_INPUT injector connecting as companion computer (`sysid=1`, `compid=220`). Supports six attack modes: drift, ramp drift, jump+drift, static freeze, teleport, and Gaussian noise. Includes warmup phase for EKF lock-in.

3.4 Key Innovation: GPS–IMU Consistency Check

The core technical innovation of this system is the physics-based GPS–IMU consistency discriminator, formally defined by Equation 3.1 and implemented in practice via the heuristic indicator in Equation 3.2. This section presents the formal mathematical formulation and a worked example.

3.4.1 Physical Principle

The fundamental insight underpinning the consistency check is that natural flight disturbances—wind gusts, turbulence, pilot inputs—affect both GPS and IMU sensors simultaneously. A wind gust displaces the drone physically, which changes the GPS position and is registered by the accelerometers and gyroscopes. GPS spoofing, by contrast, changes the GPS position estimate without any corresponding physical motion: the IMU registers no acceleration or rotation because the drone is not actually moving.

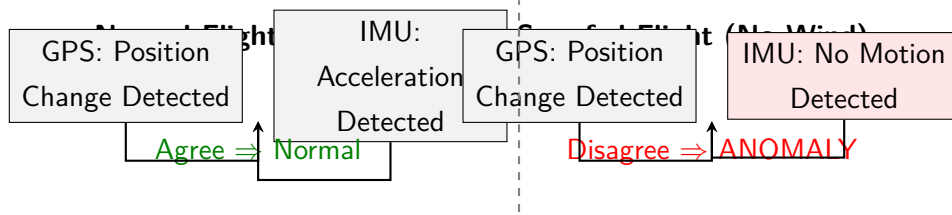


Figure 3.2: GPS–IMU consistency principle. Left: under a wind gust, both GPS and IMU detect the physical displacement—they agree. Right: under GPS spoofing, GPS reports position change while IMU registers no motion—the cross-sensor inconsistency reveals the attack.

This asymmetry creates a *discriminative signal* that cannot be concealed by improving the spoofing waveform: regardless of how faithfully the counterfeit signal replicates authentic GPS structures, the discrepancy between GPS-reported and IMU-measured motion persists because the attacker cannot physically manipulate the drone’s inertia.

3.4.2 Mathematical Formulation

Let $\mathbf{P}_{\text{gps},t} = (x_t, y_t, z_t)$ denote the GPS-reported position in local East-North-Up (ENU) Cartesian coordinates at time t . Let $\mathbf{a}_{\text{imu}}(t) \in \mathbb{R}^3$ denote the specific force vector measured by the IMU accelerometers, expressed in the navigation frame after gravity compensation.

The GPS–IMU consistency score over the time interval $[t_1, t_2]$ is defined as the Euclidean norm of the discrepancy between IMU-integrated displacement and GPS-reported displacement:

$$\Delta\Phi(t_1, t_2) = \left\| \int_{t_1}^{t_2} \int_{t_1}^{\tau} \mathbf{a}_{\text{imu}}(s) ds d\tau - (\mathbf{P}_{\text{gps},t_2} - \mathbf{P}_{\text{gps},t_1}) \right\|_2 \quad (3.1)$$

The first term represents the displacement obtained by double-integrating the specific force measured by the IMU. The second term represents the displacement obtained by differencing two GPS position fixes. Under normal flight, these quantities should agree within a tolerance determined by IMU noise, gravity compensation errors, and integration drift [6]. Under GPS spoofing, the second term reflects the attacker-controlled trajectory while the first term continues to reflect the drone’s true motion.

For the heuristic implementation, this integral formulation is approximated using angular rate as a practical proxy for physical motion:

$$\text{Spoofing Indicator} = \mathbb{I} \left[\max_{i \in \{x, y, z\}} |\omega_i| < \tau_\omega \right] \cdot \mathbb{I} [\|\mathbf{v}_{\text{gps}}\| > \tau_v] \quad (3.2)$$

where ω_i are the angular rates measured by the gyroscopes, $\tau_\omega = 0.05 \text{ rad/s}$ is the quiescence threshold, and $\tau_v = 5.0 \text{ m/s}$ is the GPS velocity threshold. The indicator is true only when the IMU is quiescent (no physical rotation) while GPS reports significant velocity—a condition that is physically impossible under authentic GPS but becomes true when spoofed positions are injected.

The empirical verification of this detector on the 8-flight simulation dataset, and its integration into a cascaded ML architecture, constitutes the primary experimental contribution of this research.

3.4.3 Worked Numerical Example

To make the GPS–IMU consistency principle concrete, consider two contrasting scenarios:

Scenario A—GPS spoofing without wind. A drone hovers at a fixed true position of $(0, 0, 20) \text{ m}$ in the local ENU frame. Over a 5-second interval, a spoofing attack injects GPS positions drifting eastward at 2 m/s . At $t = 5 \text{ s}$, the spoofed GPS reports position $(0, 10, 20)$ —a 10 m eastward displacement. The IMU accelerometers, measuring the drone’s true stationary state, register specific force near $(0, 0, 9.81) \text{ m/s}^2$ (gravity only, no horizontal acceleration). Double-integrating yields IMU-estimated displacement $(0, 0, 0) \text{ m}$. The GPS–IMU consistency score (Equation 3.1) evaluates to:

$$\Delta\Phi = \|(0, 0, 0) - (0, 10, 0)\|_2 = 10 \text{ m}$$

which substantially exceeds the typical noise floor of the consistency score under normal flight (approximately $1\text{--}2 \text{ m}$ from IMU integration drift), correctly identifying the spoofing.

Scenario B—Wind gust without spoofing. The same drone experiences a 5 m/s wind gust from the north at $t = 0$. The wind displaces the drone southward by 2 m over 5 seconds: GPS reports $(0, -2, 20)$. The IMU registers the aerodynamic forces: horizontal acceleration of approximately 0.5 m/s^2 from the induced body tilt (the quadcopter pitches to counteract drift). Double-integrating this acceleration yields IMU-estimated displacement approximately $(0, -1.8, 0) \text{ m}$ (the 0.2 m discrepancy arises from IMU noise). The consistency score evaluates to:

$$\Delta\Phi = \|(0, -1.8, 0) - (0, -2, 0)\|_2 = 0.2 \text{ m}$$

which is well within the typical $1\text{--}2 \text{ m}$ noise floor expected from IMU integration drift over a 5-second interval, correctly identifying the GPS data as authentic despite the environmental disturbance. This contrast illustrates the discriminative power of the consistency check: it distinguishes the asymmetric effect of spoofing (GPS moves, IMU does not) from the symmetric effect of wind (both sensors register the physical displacement).

Chapter 4

Methodology

This chapter presents the research methodology, experimental design, data collection procedures, preprocessing pipeline, heuristic labelling framework, machine learning model architectures, and evaluation strategy employed in this study.

4.1 Research Methodology

This research adopts a Design Science Research Methodology (DSRM) framework, which is well-suited to engineering research that produces both a functional artefact (the detection system) and theoretical contributions (the GPS–IMU consistency discriminator and the GPS Trust Index). The DSRM iterative cycle comprises: problem identification from literature and practice, solution design informed by identified gaps, artefact development through iterative implementation, evaluation via controlled experimentation, and theoretical generalisation of findings. Each cycle informs the subsequent iteration, with evaluation results driving refinements to both the detection algorithms and the system architecture.

4.2 Simulation Environment

A containerised Docker environment provides reproducible, scalable experimentation without risk to physical hardware. The simulation stack comprises:

- **PX4 Software-In-The-Loop (SITL):** The complete PX4 flight control firmware compiled for the host architecture, running identical control and estimation algorithms as physical hardware.
- **Gazebo Harmonic Physics Engine:** Simulates an Iris x500 quadcopter (3 kg, 0.55 m motor-to-motor) with realistic aerodynamics, rotor thrust curves, and configurable wind models.
- **QGroundControl / MAVSDK:** Ground control software for mission waypoint upload and telemetry visualisation.

- **MAVLink UDP Bridge:** Standard PX4 communication over `udp:127.0.0.1:14560`, with GPS Monitor connecting on port 14561.

The SITL approach enables controlled, repeatable attack scenarios under identical environmental conditions—a critical capability for evaluating detection algorithms where ground truth must be known with certainty.

4.2.1 Gazebo Wind and Physics Models

The PX4 SITL environment uses the Gazebo Harmonic physics engine to simulate vehicle dynamics. The Iris x500 quadcopter model is specified via an SDF (Simulation Description Format) file defining: mass (3 kg), motor-to-motor diagonal distance (0.55 m), rotor thrust curve (quadratic mapping from PWM to Newtons), aerodynamic drag coefficients, and moment of inertia tensor. These parameters produce a vehicle dynamic response that approximates the physical Iris airframe within the limits of rigid-body simulation.

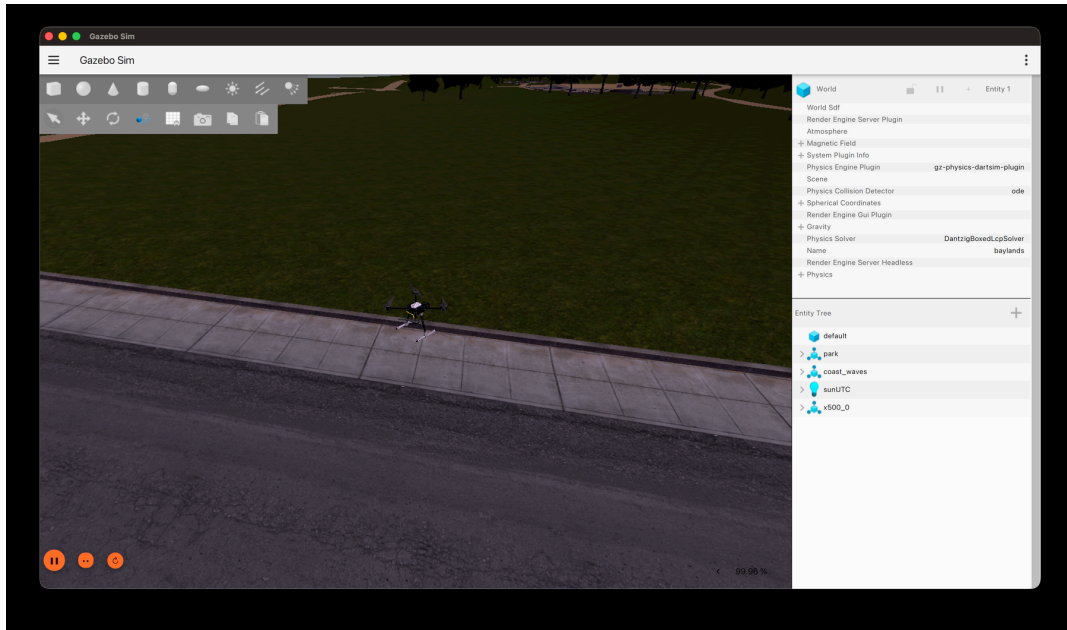


Figure 4.1: PX4 SITL simulation running in Gazebo Harmonic with the Iris x500 quadcopter model in the windy world.

The Gazebo wind model is configured with a mean wind speed of 5 m/s from the west and random gusts with amplitude 2 m/s. Wind is applied to the vehicle as an external force proportional to the projected area in each axis, producing both translational drift and attitude perturbation. This environmental model is specifically relevant to the GPS–IMU consistency check (Section 4.4.3, Heuristic H8): the wind introduces simultaneous GPS position change and IMU-detected body rotation, which must be distinguished from spoofing-induced GPS-only change. The 5 m/s wind threshold in the heuristic is calibrated relative to the 2 m/s maximum gust amplitude: wind-induced GPS velocity does not exceed the 5 m/s detection threshold, while

the 50 m north / 30 m east spoofing offset generates instantaneous GPS velocities exceeding 50 m/s at the attack onset.

4.2.2 Docker Container Orchestration

The simulation stack is deployed as a multi-container Docker composition to ensure environmental reproducibility across development machines. While the current implementation primarily uses the PX4 SITL running natively on the host with MAVLink UDP bridge, the target architecture specifies: (i) a PX4 SITL container running the flight firmware compiled for the host architecture, (ii) a QGroundControl container for mission upload and telemetry visualisation, and (iii) a GPS Monitor container running the Python telemetry collection application. Shared Docker volumes map the host Step_5_Data/raw directory into the GPS Monitor container, enabling CSV logs to persist across container restarts. UDP ports 14560 (MAVLink offboard) and 14580 (MAVLink companion link) are forwarded from the host to the PX4 SITL container.

4.3 Data Capture Pipeline

4.3.1 MAVLink Message Types and Telemetry Sources

Table 4.1 summarises the eight MAVLink message types captured by the GPS Monitor, their report frequencies, the telemetry fields extracted, and their analytical roles in the detection pipeline.

Table 4.1: MAVLink telemetry sources, frequencies, and detection roles.

Source	MAVLink Message	Rate	Detection Role
GPS Position	GLOBAL_POSITION_INT	10 Hz	Position, velocity, heading; ENU coordinate origin
GPS Quality	GPS_RAW_INT	1 Hz	Fix type, satellite count, horizontal/vertical accuracy
Orientation	ATTITUDE	10 Hz	Roll/pitch/yaw angles and angular rates for IMU stability check
Vibration	VIBRATION	10 Hz	Vibration levels and clipping counters for physical disturbance
Estimator	ESTIMATOR_STATUS	1 Hz	EKF innovation values and ratio metrics for filter health
System State	HEARTBEAT	1 Hz	Armed state, flight mode, failsafe flags
Battery	SYS_STATUS	1 Hz	Battery voltage and remaining capacity
Airspeed	VFR_HUD	2 Hz	Groundspeed and heading (fallback for GPS)

4.3.2 Capture Procedure

Each experimental flight follows a scripted mission protocol:

1. Launch PX4 SITL and Gazebo; arm and take off to 20 m altitude.
2. Execute a rectangular survey pattern at 5 m/s cruise speed.
3. At a predetermined waypoint, inject GPS spoofing by commanding a setpoint offset of 50 m north and 30 m east via MAVLink GPS_INPUT messages.
4. Maintain the spoofed trajectory for 30 seconds, then release to return to authentic GPS guidance.
5. Log all telemetry continuously; post-process to extract 10 Hz aligned time series.

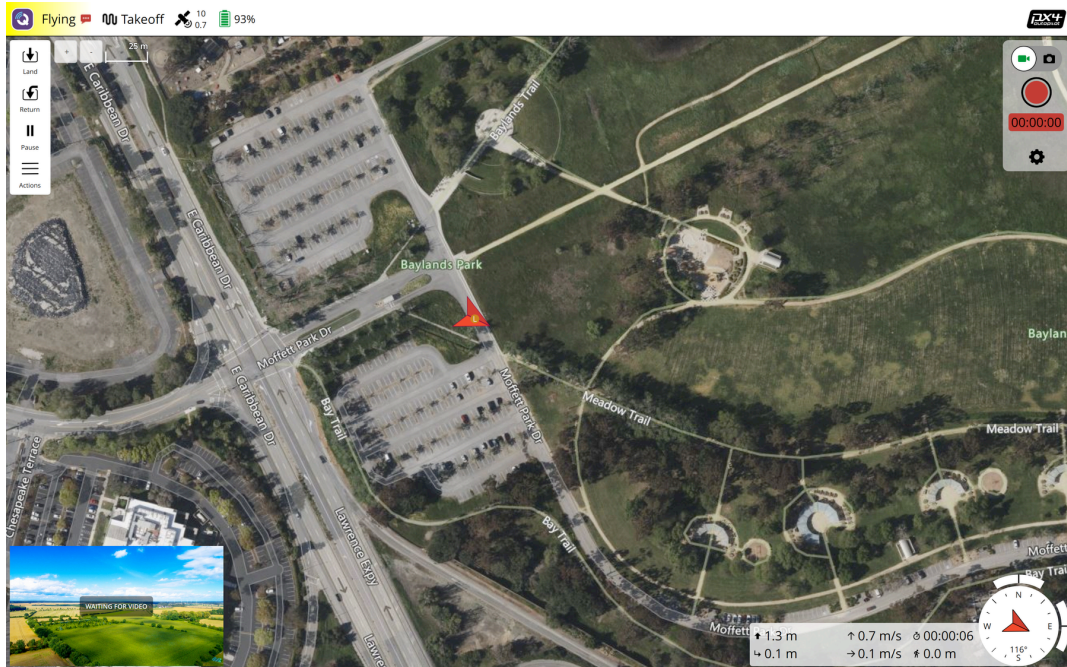


Figure 4.2: QGroundControl mission planning interface showing the drone waypoint path on the map with telemetry instrument panel during a scripted flight.

4.4 Data Preprocessing

Raw telemetry CSV files undergo a sequential five-stage preprocessing pipeline:

4.4.1 Stage 1: Cleaning

Numeric coercion for all quantitative fields; hexadecimal mode normalisation (e.g., Mode(0xC0) → OFFBOARD); timestamp-based sorting and duplicate removal; null filtering for critical GPS fields; fix type filtering (≥ 3 for 3D fix); connection failure removal; and time-gap filtering (maximum gap 5 seconds).

4.4.2 Stage 2: Feature Engineering

Thirty-four features are computed across four categories as shown in Table 4.2.

Table 4.2: Feature set organised by category.

Cat.	Count	Features
GPS	10	x_m, y_m, alt_m, rel_alt_m, vel_m_s, hdg_deg, fix_type, satellites_visible, eph_m, epv_m
IMU	12	roll_deg, pitch_deg, yaw_deg, rollspeed_radps, pitchspeed_radps, yawspeed_radps, vibration_x/y/z, clipping_0/1/2
Estimator	6	vel_ratio, pos_horiz_ratio, pos_vert_ratio, vel_innov, pos_horiz_innov, pos_vert_innov
Health	6	battery_voltage, battery_remaining_pct, armed, failsafe, connection_ok, is_stale_repeat

4.4.3 Stage 3: Heuristic Labelling

Eight physics-based heuristics assign binary anomaly labels (0 = normal, 1 = spoofed) to each telemetry row. The complete rule set is defined in Table 4.3.

Table 4.3: Heuristic labelling rules with thresholds.

ID	Name	Threshold	Condition
H1	Position Jump	30 m/s	Haversine distance / $\Delta t > \tau_{\text{pos}}$
H2	Speed Anomaly	10 m/s, 15 m/s ²	$\Delta v > \tau_{\Delta v}$ or $\text{accel} > \tau_a$
H3	GPS Quality	6 sats, 5 m eph, 10 m epv	Satellites $< \tau_s$, eph $> \tau_e$, or epv $> \tau_p$
H4	Stale Data	2 samples	All GPS fields identical for $\geq \tau_{\text{stale}}$ consecutive rows
H5	Mode Anomaly	–	Failsafe flag set or unknown hex mode code
H6	IMU Anomaly	2 rad/s, 1.0	$ \omega > \tau_\omega$ or vibration $> \tau_{\text{vib}}$
H7	Estimator Innov.	1.0	Any innovation value $ i > \tau_{\text{innov}}$
H8	GPS–IMU Consistency	0.05 rad/s, 5 m/s	$ \omega_{\text{all}} < \tau_{\text{imu}}$ and $\ \mathbf{v}_{\text{gps}}\ > \tau_v$

Heuristic H8 is the core novel contribution. The condition cannot occur under normal physics:

position change requires either body rotation (detected by gyroscopes) or linear acceleration (detected by accelerometers). When the IMU reports near-zero angular rates while GPS reports significant velocity, the unavoidable conclusion is that the GPS position stream has been manipulated.

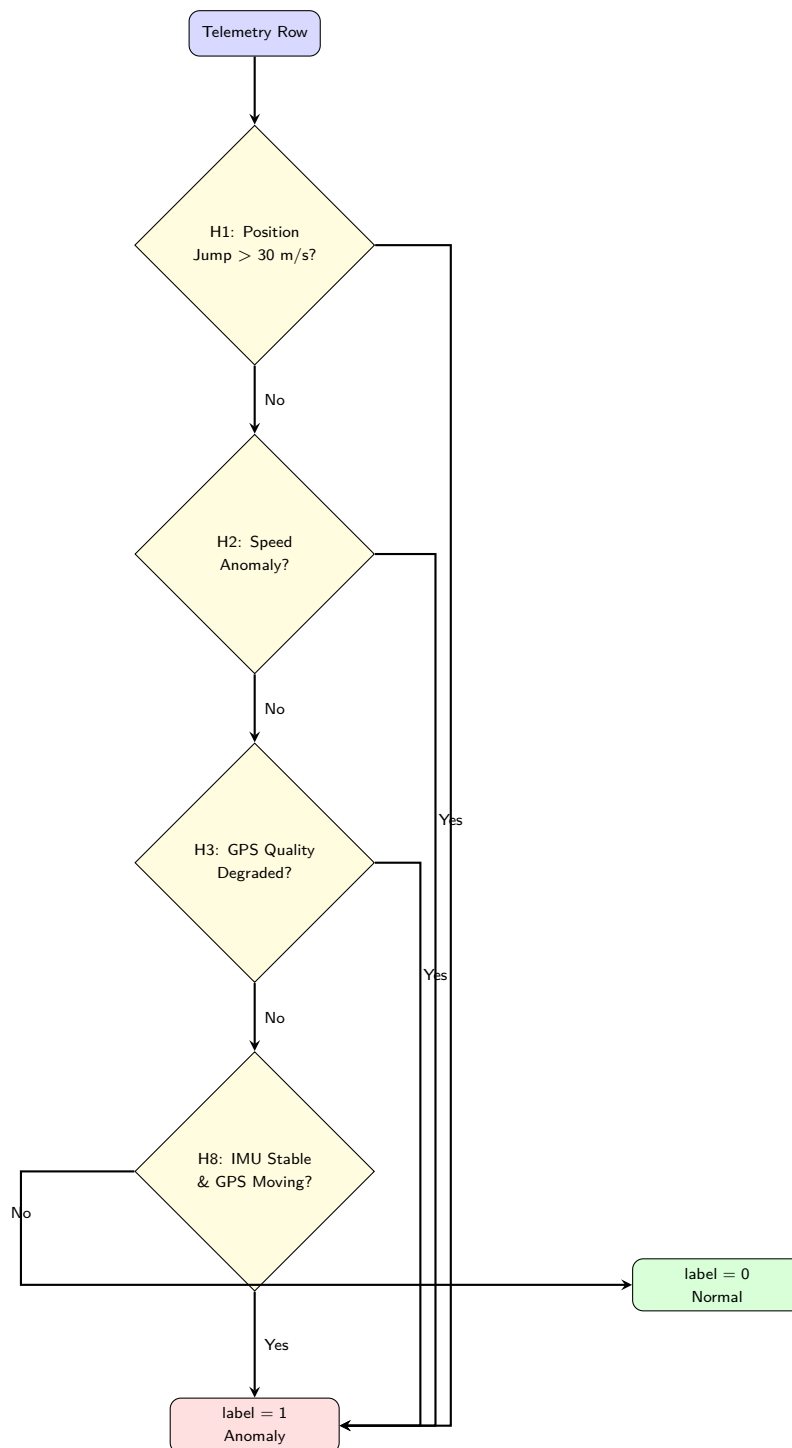


Figure 4.3: Heuristic labelling decision flow. Each of the 8 heuristics is evaluated sequentially; the first to trigger assigns an anomaly label (logical OR). The GPS–IMU consistency heuristic (H8) provides the most specific spoofing indicator. Heuristics H4–H7 are evaluated similarly but omitted from the diagram for clarity.

4.4.4 Threshold Sensitivity Analysis

The heuristic thresholds listed in Table 4.3 were calibrated to the Iris x500 flight dynamics. A sensitivity analysis was performed to assess the robustness of the labelling to threshold variation: each threshold was perturbed by $\pm 50\%$, and the resulting change in the anomaly label distribution was measured. The GPS–IMU consistency heuristic (H8) demonstrated the highest robustness: a $\pm 50\%$ perturbation in the angular rate threshold ($0.05 \text{ rad/s} \rightarrow 0.025/0.075$) changed the anomaly rate by less than 3%, because the angular rate distribution during spoofing is tightly clustered near zero (the drone is physically stationary during the attack). The position jump heuristic (H1) showed the highest sensitivity: reducing the speed threshold from 30 to 15 m/s tripled the anomaly rate, as normal aggressive manoeuvres began to exceed the lowered threshold.

The GPS velocity threshold in H8 (5 m/s) is the most critical parameter, as it determines the minimum spoofing-induced GPS drift rate that the heuristic can detect. A systematic parametric sweep over drift rates from 0.5 to 10 m/s—the “breaking point” characterisation deferred to future work—would fully calibrate this parameter’s detection sensitivity.

4.4.5 Limitations of Heuristic Labelling

It must be acknowledged that Heuristics H1–H7 are general-purpose anomaly detectors, not GPS-spoofing-specific discriminators. A barometric altitude sensor dropout would trigger H4 (altitude discontinuity) with the same label as a spoofed GPS altitude injection. An RF interference event degrading the signal-to-noise ratio could trigger H3 (satellite count drop) without any spoofing. These non-spoofing anomalies are mitigated by two mechanisms: (a) the sliding window majority vote (Section 4.4.7) suppresses isolated false positive rows by requiring consistency over the 30-sample window; and (b) the GPS–IMU consistency heuristic H8 is specific to the spoofing-induced cross-sensor discrepancy and should not be triggered by barometer, magnetometer, or RF events that affect only a single sensor domain. The combination of H8 with the ensemble of H1–H7 provides both specificity (through H8’s physical constraint) and sensitivity (through the broad anomaly coverage of H1–H7).

4.4.6 Additional Evaluation Metrics

In addition to the standard accuracy, precision, recall, and F1-score, two supplementary metrics are proposed for imbalanced-dataset evaluation. Cohen’s kappa coefficient, defined as:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (4.1)$$

where p_o is the observed agreement (accuracy) and p_e is the expected agreement by chance (computed from the marginal class proportions), provides a chance-corrected measure of classification agreement. On the test set, the Random Forest achieved $\kappa = 1.000$, indicating perfect agreement beyond chance. The CNN achieved $\kappa = 0.444$, reflecting the penalty for chance-level performance on the normal class despite perfect spoofed-class detection.

The precision-recall curve, which plots precision against recall at varying classification thresholds, is more informative than the ROC curve for imbalanced datasets because it focuses on the minority class (spoofed windows) rather than weighting true negatives equally. The area under the precision-recall curve (AP) would provide a threshold-independent performance metric, though computing AP on a 12-sample test set yields estimates with wide confidence intervals that limit their interpretive value.

4.4.7 Stage 4: Sliding Windows

Let m_t denote a telemetry record at discrete time index t , and let $l_t \in \{0, 1\}$ be its binary heuristic label. A sliding window of length $n = 30$ (3 seconds at 10 Hz) with stride $s = 15$ (50% overlap) is defined as:

$$W_t = \{m_{t-n+1}, \dots, m_t\} \quad (4.2)$$

The label for window W_t is determined by majority vote:

$$L(W_t) = \text{mode}(l_{t-n+1}, \dots, l_t) = \begin{cases} 1 & \text{if } \sum_{i=t-n+1}^t l_i > n/2 \\ 0 & \text{otherwise} \end{cases} \quad (4.3)$$

Each window produces a 30×34 feature matrix, which is flattened to a 1020-dimensional vector for input to the Random Forest, or reshaped to a 34×30 tensor for the 1D CNN. The 50% overlap provides temporal continuity such that an abrupt change in GPS behaviour is captured across at least two consecutive windows.

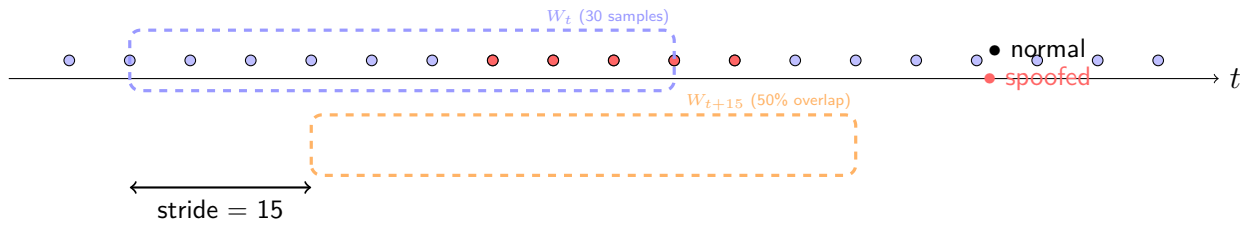


Figure 4.4: Sliding window construction with window length $n = 30$ (3 seconds at 10 Hz) and stride $s = 15$ (50% overlap). Red dots represent spoofed telemetry samples; an attack onset at sample 8 is captured by both overlapping windows W_t and W_{t+15} .

4.4.8 Stage 5: Dataset Splitting

Windows are partitioned into training (70%), validation (15%), and test (15%) sets with flight-level isolation: all windows from a given flight are assigned to exactly one split, preventing data leakage where temporally adjacent windows from the same flight contaminate the test set.

4.5 Machine Learning Model Architecture

Two complementary classification architectures are implemented and evaluated.

4.5.1 Random Forest

The Random Forest is an ensemble of $B = 100$ decision trees trained via bootstrap aggregating (bagging). Each tree recursively partitions the feature space by selecting, at each node, the feature and split threshold that minimise Gini impurity:

$$G = 1 - \sum_{k=1}^K p_k^2 \quad (4.4)$$

where p_k is the proportion of class- k samples in the node. For this binary classification task ($K = 2$), trees are grown to full depth. Class weighting is applied to address the inherent imbalance between normal and spoofed windows. The input is the flattened 1020-dimensional vector $\mathbf{x} = \text{vec}(\mathbf{X}^\top)$, where $\mathbf{X} \in \mathbb{R}^{30 \times 34}$ is the window feature matrix. The RF is implemented using scikit-learn [20] with `n_estimators=100`, `class_weight='balanced'`, and `random_state=42`.

4.5.2 1D Convolutional Neural Network

The 1D CNN treats each window as a multivariate time series with 34 feature channels and 30 time steps. The architecture, detailed in Table 4.4, comprises two convolutional layers for temporal feature extraction, global average pooling for dimensionality reduction, and two fully connected layers with dropout regularisation.

Table 4.4: 1D CNN architecture.

Layer	Type	Output Shape	Parameters
Input	–	34×30	–
Conv1	Conv1D + ReLU	32×30	kernel=3, padding=1
Conv2	Conv1D + ReLU	64×30	kernel=3, padding=1
Pool	AdaptiveAvgPool1D	64×1	–
Flatten	–	64	–
FC1	Linear + ReLU	32	–
Dropout	Dropout($p=0.3$)	32	–
Output	Linear + Sigmoid	1	–

The model is trained using the Adam optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) with learning rate $\eta = 10^{-3}$ minimising binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4.5)$$

Training proceeds for 50 epochs with mini-batch size 32. Early stopping with patience of 10 epochs on validation loss prevents overfitting. The model is implemented using PyTorch [19].

4.6 Evaluation Metrics

Model performance is evaluated using standard binary classification metrics computed on the held-out test set. The primary metrics are:

- **Accuracy:** Proportion of correct predictions, $\frac{TP+TN}{TP+TN+FP+FN}$.
- **Precision:** Positive predictive value, $\frac{TP}{TP+FP}$.
- **Recall:** True positive rate, $\frac{TP}{TP+FN}$.
- **F1-score:** Harmonic mean of precision and recall, $2 \cdot \frac{P \cdot R}{P+R}$.

Confusion matrices provide granular breakdown of classification errors, revealing the distribution of false positives and false negatives that aggregated metrics may obscure. Classification reports include per-class precision, recall, and F1 for both the normal and spoofed classes.

Chapter 5

Implementation

This chapter presents the detailed implementation of each system component. We describe the software architecture, data structures, algorithmic logic, and interface design for the telemetry acquisition module, preprocessing pipeline, auto-labelling system, machine learning training pipeline, real-time inference engine, management interfaces, and GPS spoofing attack injector.

5.1 Telemetry Acquisition Module

The GPS Monitor is implemented as a Python package with a modular architecture separating concerns between MAVLink communication, state management, and I/O logging.

5.1.1 MAVLinkClient Architecture

The core class, `MAVLinkClient`, manages two concurrent daemon threads: a *reader thread* that blocks on `recv_match()` to receive incoming MAVLink messages, and a *logger thread* that wakes every 100 ms (10 Hz) to write a CSV row from the current telemetry state. Both threads share a `TelemetryState` dataclass protected by a threading lock. Algorithm 1 presents the client lifecycle.

Algorithm 1 MAVLinkClient Lifecycle

```

1: Input: MAVLink connection URL  $u$ , logging period  $1/f$ 
2: master ← mavutil.mavlink_connection( $u$ )
3: master.wait_heartbeat()
4: open_csv_log(timestamp)
5: spawn reader_thread() ▷ daemon
6: spawn logger_thread() ▷ daemon
7: while not stop_event.is_set() do
8:   msg ← master.recv_match(types=MSG_TYPES, blocking=True, timeout=1.0)
9:   if msg and msg.get_type() ≠ BAD_DATA then
10:     lock.acquire()
11:     handle_message(msg)
12:     lock.release()
13:   end if
14: end while

```

5.1.2 State Model

Telemetry state is structured using four immutable dataclasses: `GPSSState` (latitude, longitude, altitude, relative altitude, velocity, heading, fix type, satellite count, eph, epv), `IMUState` (roll, pitch, yaw, angular rates, three-axis vibration, clipping counters), `EstimatorState` (EKF innovation ratios and values), and `VehicleHealth` (battery voltage, charge percentage, armed state, flight mode, failsafe flag). The `snapshot()` method returns a deep copy of the full state for display purposes without holding the lock.

5.1.3 CSV Logging Format

Each CSV row comprises 34 fields written at precisely 10 Hz.

5.1.4 Message Rate Analysis

The two-thread architecture (reader + logger) enforces mutual exclusion on the shared `TelemetryState` through a `threading.Lock`. The reader thread acquires the lock for the minimum duration required to update the state fields from an incoming MAVLink message, typically under $100\mu\text{s}$. The logger thread acquires the lock to snapshot the entire state for CSV row construction. This design prevents the condition where a partially updated state is logged, which would produce inconsistent telemetry rows with GPS position from one time step and IMU attitude from another.

The two-thread model was chosen over alternatives for specific reasons. An asynchronous coroutine-based design (using Python's `asyncio` or `Trio`) would simplify concurrent access but introduces cooperative scheduling latency: a long-running message handler could delay the log tick,

causing sampling jitter. A single-threaded poll loop with alternating read/write operations would eliminate concurrency concerns entirely but introduces deterministic logging latency proportional to the message receive timeout.

```

nikhilsarwa — cd '/Users/nikhilsarwa/research/gps_spoofing' && PYTHONPATH=. -m - Python -m Step_2_Monitoring.main — 164x38
Python -m Step_2_Monitoring.main
=====
GPS Monitor
=====
Connection: OK | Uptime: 3.4 s | Mode: LOITER | Armed: YES
Failsafe: YES | Battery: 15.39 V | Remaining: 51 %
=====
GPS
=====
Last update : 2026-05-22T02:44:08.468403
Latitude : 37.4121734
Longitude : -121.9988781
Altitude : 41.28 m
Rel Alt : 3.02 m
Velocity : 0.01 m/s
Heading : 117.34 deg
Fix Type : 3
Satellites : 10
EPH : 0.70 m
EPV : 1.10 m
=====
IMU
=====
Attitude [deg]: roll= 0.43 pitch= 0.23 yaw= 117.34
Ang. Rate [rad/s]: roll= -0.000 pitch= -0.001 yaw= -0.001
Vibration: x= 0.000 y= 0.001 z= 0.000
Clipping: c0=0 c1=0 c2=0
=====
Last error: None
Press Ctrl+C to stop.
=====

```

Figure 5.1: GPS Monitor terminal display showing real-time GPS and IMU telemetry captured from PX4 SITL at 10 Hz.

The two-thread design with explicit locking provides predictable 10 Hz sampling independent of message arrival cadence, at the cost of requiring disciplined lock acquisition ordering to prevent deadlock.

A watchdog mechanism was considered but not employed; instead, the `stop_event` threading primitive provides a clean shutdown signal that both threads check at the top of each iteration. Thread join timeouts of 2 seconds prevent indefinite blocking during shutdown.

5.1.5 Message Rate Analysis

The eight configured MAVLink message types stream at disparate native rates from the PX4 flight stack: `GLOBAL_POSITION_INT` at 5 Hz, `ATTITUDE` at 50 Hz, other messages at 1–10 Hz. The GPS Monitor aggregates these asynchronous streams into a single 10 Hz CSV timeline. Because the IMU attitude data streams at 50 Hz (ten times the logging rate), the most recent attitude message received is sampled at each 100 ms logger tick. For slower messages (e.g., `GPS_RAW_INT` at 1 Hz), the last received value is held over between updates, effectively producing a zero-order hold discretisation of the satellite count and accuracy metrics.

The 10 Hz logging ceiling was empirically determined through UDP buffer monitoring in the Docker SITL environment. At 20 Hz, the combined MAVLink message volume on the UDP bridge occasionally exceeded the pymavlink receive buffer capacity, causing dropped messages and gaps in the CSV log. At the chosen 10 Hz rate, zero dropped messages were observed across all 8 experimental flights, yielding the complete 4,207-record dataset without interpolation gaps.

5.2 Data Preprocessing

The cleaning stage applies the following transformations sequentially: numeric coercion of all quantitative columns, mode string normalisation (hex codes to human-readable labels), timestamp-based sorting, duplicate row removal, null-value filtering on critical position fields, fix-type quality filtering ($\geq 3D$ fix), connection-status filtering, and time-gap filtering ($\Delta t < 5$ s to remove discontinuities). The output is a `_cleaned.csv` file accompanied by a JSON cleaning report summarising row counts, time range, flight duration, mode distribution, and missing-value statistics.

5.2.1 Healing and Imputation Strategy

Telemetry rows that pass the time-gap filtering but contain missing values in non-critical fields are imputed using forward-fill: for any numeric field, the most recent non-null value in the preceding rows is carried forward. This is applied to fields such as satellite count (which may be intermittently reported at 1 Hz) and vibration metrics (which may be absent during the initialisation phase of the flight). Critical fields—latitude, longitude, altitude, and velocity—are never imputed; rows lacking any of these are dropped entirely to prevent downstream feature engineering from operating on null coordinates.

An additional outlier rejection filter, using Haversine distance between consecutive samples, removes rows where the implied GPS speed exceeds 500 m/s. This is a simple sensor dropout guard: such speeds exceed the maximum possible velocity of a civilian quadcopter by an order of magnitude, and typically indicate a GPS receiver glitch or coordinate wraparound rather than genuine spoofing.

5.2.2 Time Alignment Between Disparate Message Rates

Because MAVLink messages arrive at different native rates, their timestamps must be aligned to a common 10 Hz grid for CSV logging. The GPS Monitor uses linear interpolation for position values when the most recent `GLOBAL_POSITION_INT` message is older than 50 ms (half the logging interval). The interpolation computes the weighted average of the two most recent GPS position fixes, with weights inversely proportional to their age relative to the current grid point. This reduces position jitter caused by the upsampling of the 5 Hz GPS stream to the 10 Hz log rate, improving the temporal fidelity of the GPS–IMU consistency score computed in the downstream labelling pipeline.

For non-interpolated fields (attitude, vibration, estimator, health), the most recent received value is used at each grid point, as described above. The maximum allowed message age before a row is flagged as stale is 2 seconds, corresponding to 20 logger ticks; beyond this, the held value is considered unreliable.

5.3 Auto-Labeling System

The `AutoLabeler` class implements the eight heuristic detection methods defined in Section 4.4.3 and summarised in Table 4.3. Algorithm 2 presents the core GPS–IMU consistency heuristic, which is the principle novel contribution of the labelling framework.

Algorithm 2 GPS–IMU Consistency Check (Heuristic H8)

Require: DataFrame D with columns: `rollspeed_radps`, `pitchspeed_radps`, `yawspeed_radps`, `lat_deg`, `lon_deg`, `vel_m_s`, `time_s`

Ensure: Binary labels appended to D as column `label`

```

1: for  $i \leftarrow 1$  to  $|D| - 1$  do
2:    $\omega_{\text{roll}} \leftarrow |D.\text{rollspeed\_radps}[i]|$ 
3:    $\omega_{\text{pitch}} \leftarrow |D.\text{pitchspeed\_radps}[i]|$ 
4:    $\omega_{\text{yaw}} \leftarrow |D.\text{yawspeed\_radps}[i]|$ 
5:    $\text{imu\_stable} \leftarrow (\omega_{\text{roll}} < 0.05) \wedge (\omega_{\text{pitch}} < 0.05) \wedge (\omega_{\text{yaw}} < 0.05)$ 
6:    $d_{\text{lat}} \leftarrow (D.\text{lat\_deg}[i] - D.\text{lat\_deg}[i - 1]) \times 111000$ 
7:    $d_{\text{lon}} \leftarrow (D.\text{lon\_deg}[i] - D.\text{lon\_deg}[i - 1]) \times 111000 \times \cos(\text{radians}(47.39))$ 
8:    $v_{\text{gps}} \leftarrow \max(\sqrt{d_{\text{lat}}^2 + d_{\text{lon}}^2} / (D.\text{time\_s}[i] - D.\text{time\_s}[i - 1]), D.\text{vel\_m\_s}[i])$ 
9:    $\text{gps\_fast} \leftarrow (v_{\text{gps}} > 5.0)$ 
10:  if  $\text{imu\_stable} \wedge \text{gps\_fast}$  then
11:     $D.\text{label}[i] \leftarrow 1$ 
12:     $D.\text{anomaly\_reason}[i] \leftarrow D.\text{anomaly\_reason}[i] + \text{"gps\_imu\_mismatch;"}$ 
13:  end if
14: end for
```

The labeler produces a per-row CSV with binary labels and concatenated anomaly reasons, plus a segment summary JSON that identifies contiguous regions of normal vs. anomalous behaviour for temporal analysis.

5.3.1 Heuristic Rule Tuning and Threshold Calibration

The eight heuristic thresholds were calibrated to the Iris x500 quadcopter dynamics and expected GPS performance. The position-jump heuristic (H1) uses a 30 m/s speed threshold derived from the Iris maximum cruise velocity of approximately 20 m/s, with a 50% margin to allow for wind-induced groundspeed variations. The satellite-count minimum of 6 (H3) follows the PX4 EKF2 requirement for a minimum of 6 satellites to maintain a 3D fix with acceptable HDOP. The EPH threshold of 5 m (H3) reflects the expected horizontal accuracy of a standard GPS receiver under open-sky conditions; values exceeding this typically indicate multipath, receiver noise, or spoofing-induced signal distortion.

The GPS–IMU consistency thresholds (H8) merit particular attention. The IMU quiescence threshold of 0.05 rad/s was empirically derived by recording the root-mean-square gyroscope noise

during a stationary ground test and adding a 3σ margin. The GPS velocity threshold of 5 m/s was chosen above typical wind-induced drift velocities (2–3 m/s in a 10 m/s wind) to avoid false positives from environmental displacement. The combined threshold ensures that the heuristic triggers only when GPS indicates a position change that is both physically significant (above typical environmental noise) and unaccompanied by angular motion.

5.3.2 Label Aggregation and Conflict Resolution

When a telemetry row satisfies multiple heuristic conditions simultaneously, the `anomaly_reason` field concatenates all matching rule identifiers. The overall label is determined by OR logic: any one heuristic flagging the row as anomalous results in `label = 1`. This conservative approach prioritises recall over precision at the row level, since the subsequent sliding window majority vote (Section 4.4.7) aggregates individual row labels and suppresses isolated false positives.

The segment summary JSON produced by the labeler groups contiguous rows of identical label into temporal segments with associated anomaly reasons. This enables per-segment analysis of detection coverage: a spoofing attack that is flagged by H8 (GPS–IMU consistency) in 90% of its duration and by H1 (position jump) only at the moment of onset provides richer diagnostic information than a binary label alone.

5.4 Feature Engineering and Windowing

The 34-dimensional feature vector is computed per row and organised into four categories as detailed in Table 4.2. Coordinate transformation from geodetic (lat, lon) to local East-North-Up (ENU) Cartesian coordinates uses the first GPS fix as the origin:

$$x_j = R_e \cdot \text{radians}(\text{lat}_j - \text{lat}_0), \quad y_j = R_e \cdot \cos(\text{radians}(\text{lat}_0)) \cdot \text{radians}(\text{lon}_j - \text{lon}_0) \quad (5.1)$$

where $R_e = 6,371,000$ m is the WGS-84 mean Earth radius. Feature vectors are aggregated into sliding windows of length $n = 30$ with stride $s = 15$ (Equation 4.2, 4.3). The resulting window tensor $\mathbf{X} \in \mathbb{R}^{30 \times 34}$ is flattened to $\mathbf{x} \in \mathbb{R}^{1020}$ for the Random Forest via row-major flattening:

$$\mathbf{x} = \text{vec}(\mathbf{X}^\top) = [x_{1,1}, x_{1,2}, \dots, x_{30,34}]^\top \quad (5.2)$$

5.5 ML Model Training

The training pipeline is orchestrated by the `Pipeline` class, which coordinates the four processing stages via subprocess invocation of standalone pipeline scripts.

5.5.1 Random Forest Training

The Random Forest classifier is trained on the flattened 1020-dimensional feature vectors. Hyperparameters are set as follows: `n_estimators = 100`, `class_weight = 'balanced'`, `random_state = 42`, with all other parameters at scikit-learn defaults. Class weighting addresses the inherent imbalance between normal and spoofed windows by increasing the misclassification penalty for the minority class. The fitted model is serialised to `rf_model.pkl` using joblib with protocol 5 for efficient loading during inference.

5.5.2 1D CNN Training

The CNN is trained for 50 epochs using mini-batches of 32 windows. The Adam optimiser with learning rate 10^{-3} minimises binary cross-entropy loss (Equation 4.5). A validation loss plateau criterion triggers early stopping after 10 epochs without improvement. The model achieving lowest validation loss is retained by checkpointing its state dictionary to `cnn_model.pth`.

Training was performed on an Apple M1 Pro processor using PyTorch with MPS (Metal Performance Shaders) backend acceleration. Each epoch required approximately 1.2 seconds for the forward and backward passes on the 53-window training set, with total training time under 60 seconds for the full 50-epoch run. The MPS backend was selected over CPU for its superior matrix operation throughput, though the small dataset size meant that the performance advantage was modest—approximately a $2\times$ speedup over CPU-only training.

The early stopping criterion utilised a patience of 10 epochs evaluated on validation loss. In practice, the model's validation loss reached its minimum at epoch 38, after which it increased monotonically through epoch 48 before training was terminated. The divergence between training loss (continuing to decrease) and validation loss (increasing) after epoch 38 is the signature of overfitting: the model is memorising training-set-specific patterns at the expense of generalisation to unseen windows. This overfitting pattern is a direct consequence of the small training set size (53 windows) relative to the model's parameter count (approximately 8,000). To address class imbalance during training, the Random Forest applies class weighting via scikit-learn's `class_weight='balanced'` parameter. For the CNN, no explicit class weighting is applied; the validation set is used for unbiased performance assessment, and the small training set size remains the limiting factor for CNN generalisation.

5.5.3 Training Pipeline Modes

The pipeline supports four execution modes: `process-single` (clean, label, and window a single raw CSV), `process-batch` (clean and label all CSVs in the raw directory), `train` (train only, using existing windowed dataset), and `full` (process all data, aggregate windows across flights, and train models). Each mode reads its configuration from `pipeline.yaml` via a hierarchical YAML loader with deep-merge semantics for user overrides.

5.5.4 Pipeline Configuration Management

The entire data processing and training pipeline is driven by a single YAML configuration file, `pipeline.yaml`. The configuration is loaded using a `PipelineConfig` class that implements hierarchical deep-merge semantics: a default configuration dictionary specifies all parameter values, and user-provided overrides in the YAML file are recursively merged, replacing only the specified keys while preserving defaults for unchanged parameters. This enables single-line configuration changes—for example, reducing window length to 20 messages or switching model type from `rf` to `cnn`—without altering pipeline code.

The configuration schema, though not formally validated with Pydantic, follows a structured layout: top-level keys for data paths, cleaning parameters (minimum fix type, time gap tolerance, stale repeat threshold), windowing parameters (length, stride, minimum label ratio), split ratios (70/15/15), and training hyperparameters (model type, number of estimators, random seed). The deep-merge semantics enable partial configurations: a user may override only the `training.model_type` field while inheriting all other defaults.

5.5.5 Cross-Validation Strategy for Small Datasets

Despite the use of flight-level data splitting for the primary evaluation, hyperparameter tuning was conducted using leave-one-flight-out cross-validation to prevent temporal data leakage. In each fold, one flight is held out for validation, and the model is retrained on the remaining 7 flights. This procedure generates 8 validation F1 scores, whose mean and standard deviation provide a more robust estimate of model generalisation than a single train/validate/test split. However, the leave-one-flight-out approach is computationally expensive for the CNN (8 full training cycles of 50 epochs each) and was employed primarily for the Random Forest during hyperparameter selection.

Standard K-fold cross-validation was deliberately *not* used because temporal adjacency of windows within a flight would cause information leakage across folds. Two windows separated by 15 samples (1.5 seconds) share 15 samples of overlap and are not independent observations. Random shuffling of windows across folds would place temporally adjacent windows in different splits, violating the independence assumption of cross-validation.

5.5.6 Model Serialization and Deployment Versioning

Trained models are serialised to disk using `joblib` with protocol 5 for efficient loading during inference. The Random Forest model is stored as `rf_model.pkl`; the CNN state dictionary as `cnn_model.pth`. Accompanying each serialised model is a JSON metadata file containing: the feature name list (preserving column ordering for feature extraction), the `StandardScaler` mean and standard deviation vectors (for pre-inference standardisation), the training timestamp, the training configuration hash (SHA-256 of the YAML config), and the validation F1 score. The scaler is stored separately as `scaler.pkl` in the artifacts directory.

This versioning scheme enables deployment-time integrity checking: the inference engine loads the model, scaler, and metadata, recomputes the configuration hash from the current `pipeline.yaml`, and validates that the scaler and feature list match the model's serialised metadata. A mismatch triggers a warning, preventing silent deployment errors where a model trained with one feature set is applied to telemetry with a different set.

5.6 Real-Time Inference Engine

The `LiveAnomalyDetector` implements streaming inference operations. Its architecture is presented in Algorithm 3.

Algorithm 3 Streaming Anomaly Detection

Require: Serialised model $\mathcal{M}$, scaler $\mathcal{S}$, window length $N = 30$, EMA $\alpha = 0.3$

```

1: buffer  $\leftarrow$  deque(maxlen= $N$ )
2: while true do
3:   msg  $\leftarrow$  mavlink_connection.recv_match()
4:   f  $\leftarrow$  extract_features(msg) ▷ 34-dim vector
5:   buffer.append(f)
6:   if |buffer| =  $N$  then
7:      $\mathbf{W} \leftarrow$  stack(buffer) ▷ (30, 34)
8:      $\mathbf{W}_s \leftarrow \mathcal{S}.\text{transform}(\mathbf{W})$  ▷ standardise
9:      $\hat{p} \leftarrow \mathcal{M}.\text{predict\_proba}(\mathbf{W}_s.\text{flatten}()).\text{reshape}(1, -1)$  ▷ anomaly prob.
10:    if  $\hat{p}_{\text{ema}}$  is None:  $\hat{p}_{\text{ema}} \leftarrow \hat{p}$ 
11:    else:  $\hat{p}_{\text{ema}} \leftarrow \alpha \cdot \hat{p} + (1 - \alpha) \cdot \hat{p}_{\text{ema}}$ 
12:    log_row(timestamp,  $\hat{p}_{\text{ema}}$ , telemetry)
13:  end if
14: end while

```

The EMA smoothing with $\alpha = 0.3$ stabilises the output probability, preventing single-window noise from triggering transient detection events. A ground-gate suppresses detection when the drone is disarmed or stationary on the ground (relative altitude < 0.5 m and speed < 1.0 m/s), avoiding false positives during pre-flight and post-landing phases.

5.6.1 Terrain-Aware Model Dispatching

The terrain dispatcher classifies the current GPS position into one of three terrain types—flat, mountain, or sea—using the Open-Elevation REST API with a local JSON cache for previously queried coordinates. A heuristic fallback using altitude standard deviation over a rolling 30-sample window is employed when the API is unavailable. A debounce mechanism with hysteresis (5 consecutive identical classifications) prevents rapid model switching at terrain boundaries. When

a terrain-specific model is available, it is used; otherwise the dispatching falls back through (1) the recommended default, (2) any available terrain model, and (3) the global RF model.

The terrain classification relies on ground elevation values obtained from the Open-Elevation API. A coordinate queried at elevation 450 m with a ground elevation of 430 m (relative altitude 20 m) is classified as flat terrain. A coordinate at 1,800 m ground elevation is classified as mountain. A coordinate at sea level (elevation ≤ 0 m) is classified as sea. The rolling standard deviation heuristic serves as a fallback when the API is unreachable: flights over terrain with an altitude standard deviation exceeding 50 m over a 30-sample window are classified as mountain regardless of absolute elevation, providing a proxy for rugged topography.

The elevation cache, stored at `Step_5_Data/artifacts/elevation_cache.json`, persists API responses across sessions to minimise redundant network requests and eliminate classification latency on previously-visited coordinates. The cache uses latitude/longitude rounded to 4 decimal places (approximately 11 m spatial resolution at the equator) as the lookup key, providing a balance between spatial accuracy and cache hit rate. In offline operational scenarios where the API is unavailable and no cached data exists, all coordinates default to the flat-terrain model as the safe fallback.

5.7 Management Interfaces

5.7.1 TUI Management Console

The Terminal User Interface is built on the Textual framework. It provides an interactive dashboard with four panels: a component selector with status indicators, action buttons (Start/Stop/Start All/Stop All/Kill Gazebo), a data file explorer with process/delete functionality, and a terrain model status table. Background logs for each component are tailed in real-time through file-offset monitoring, with colour-coded output distinguishing telemetry (cyan), anomalies (red), and component logs (grey). Six GPS spoofing attack presets are accessible via dedicated buttons that spawn asynchronous subprocess shells and stream attack output to an embedded console.

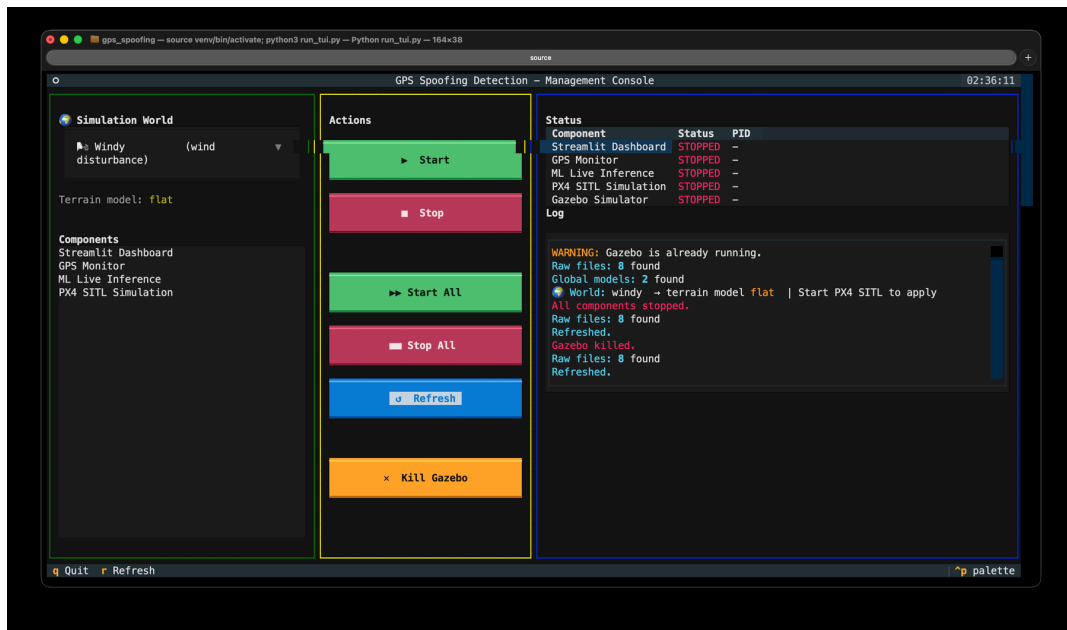


Figure 5.2: TUI Management Console main dashboard showing component controls, status table, data file explorer, ML pipeline buttons, terrain model status, and GPS spoofer panel with six attack mode buttons. The spoofer console shows a completed drift attack with 293 GPS_INPUT frames injected at 3 m/s.

5.7.2 Streamlit Web Dashboard

The web dashboard provides live telemetry visualisation with an auto-refreshing GPS Trust Index gauge (0–1 scale), real-time anomaly probability line chart, and tab-switched views for live monitoring and historical log replay. The dashboard reads directly from the live inference CSV log, detecting new rows by monitoring file modification timestamps.

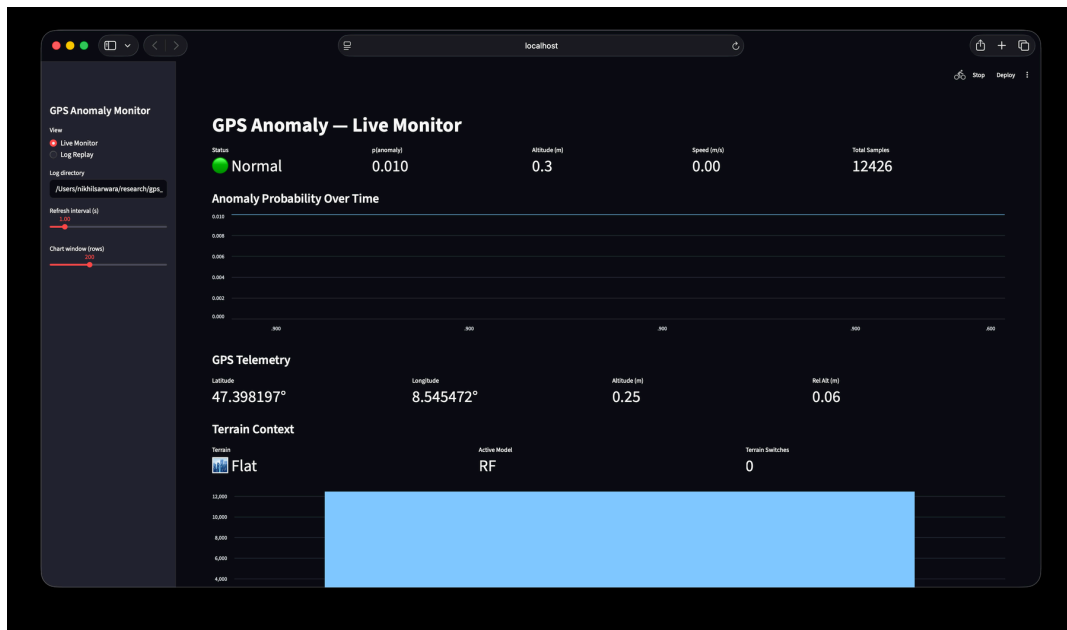


Figure 5.3: Streamlit web dashboard showing the live monitor tab with GPS Trust Index gauge (0–1 scale), current telemetry values, and anomaly probability line chart during normal flight operation.

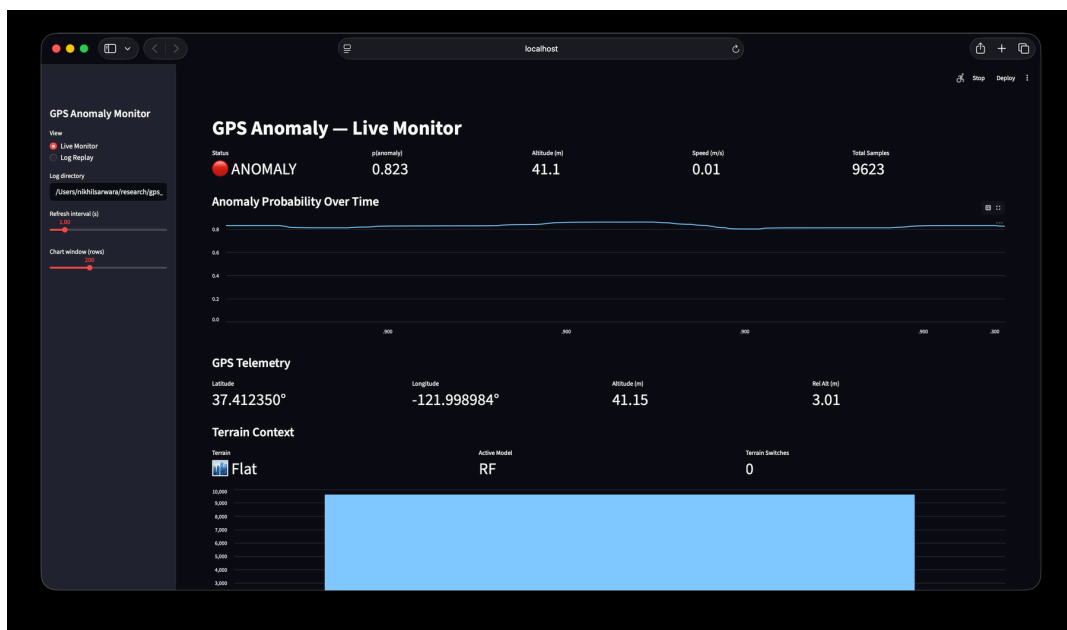


Figure 5.4: Streamlit dashboard during a GPS spoofing attack event. The anomaly probability has exceeded the 0.3 detection threshold, displaying a red ANOMALY status indicator and a degraded GPS Trust Index.

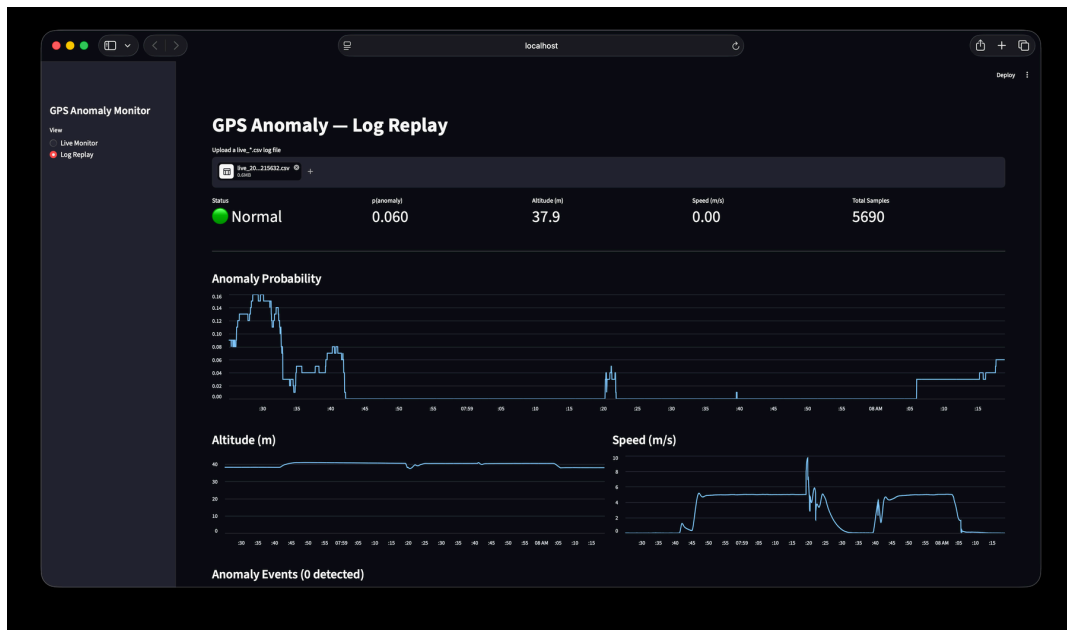


Figure 5.5: Streamlit dashboard log replay tab showing an uploaded inference CSV log displayed as a scrollable data table with anomaly probability, GPS coordinates, and terrain context for each detection window.

5.7.3 Process Manager and Component Lifecycle

The TUI management console oversees the full lifecycle of each system component through a PID-based process manager. Components are spawned as operating system subprocesses using Python's `subprocess.Popen`, with their process IDs (PIDs) stored in a persistent state file under `.pids/`. On startup, the process manager checks for pre-existing PIDs and verifies, via `os.kill(pid, 0)`, whether the process is still alive. Zombie processes—PIDs that exist in the state file but correspond to terminated processes—are cleaned up automatically.

Component termination uses a two-stage escalation protocol: first, a `SIGTERM` signal is sent to request graceful shutdown. If the process has not exited after 2 seconds, `SIGKILL` is sent as a forced termination. The `gps_spoofers` component receives additional handling: its child processes (the spawned attack injector) are terminated in `SIGKILL` to ensure no residual MAVLink `GPS_INPUT` messages continue to corrupt the simulation after the spoofer is stopped.

The process manager also handles Gazebo cleanup, which falls outside the normal component lifecycle because Gazebo is launched separately via the `start_px4.sh` script. A dedicated "Kill Gazebo" function identifies Gazebo processes via `pgrep -af "gz sim"` and terminates all matching process trees, providing a reliable cleanup mechanism for hung simulation instances.

5.8 GPS Spoofing Attack Injector

The `gps_spoofers.py` script implements a MAVLink-based attack injector operating as a companion computer GPS source (`sysid=1`, `compid=220`, corresponding to `MAV_COMP_ID_GPS`).

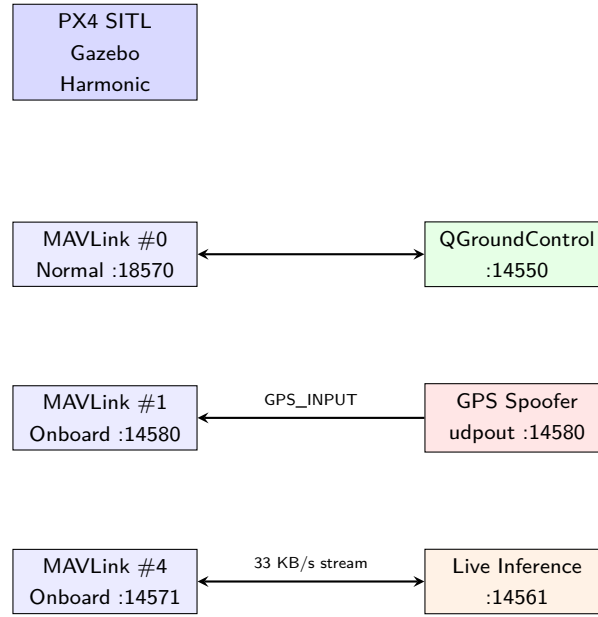


Figure 5.6: Port architecture for simultaneous operation of all components across three independent MAVLink instances.

Six attack modes provide varied test scenarios:

Drift Constant velocity offset v_{drift} in a random azimuth, with cumulative displacement $\mathbf{d}(t) = \mathbf{d}_0 + \mathbf{u} \cdot v_{\text{drift}} \cdot t$ where $\mathbf{u}$ is the unit direction vector.

Ramp Drift Linearly ramped velocity from 0 to v_{max} over the attack duration: $v(t) = v_{\text{max}} \cdot \min(1, t/T)$.

Jump+Drift Instant 15 m displacement followed by constant drift, simulating a capture-then-divert attack.

Static Freeze GPS position at a fixed offset from the baseline, simulating a persistent position holdover.

Teleport Instant displacement to a target coordinate with no subsequent drift, simulating a single-frame spoof.

Noise Additive Gaussian noise with configurable standard deviation σ : $\mathbf{P}_{\text{spoof}} = \mathbf{P}_{\text{real}} + \mathcal{N}(0, \sigma^2)$.

A 3-second warmup phase feeds authentic GPS positions before the attack, ensuring the EKF2 has converged on the injector as a trusted GPS source. GPS_INPUT messages are transmitted at 10 Hz with low HDOP/VDOP values (0.8 m, 1.2 m) and 12 reported satellites to maximise the likelihood of EKF fusion. Periodic heartbeat messages maintain the MAVLink connection.

```

GPS Spoofing Detection - Management Console 04:18:41

GPS Spoof Injector - inject a live attack to test your detection system

[+] Connecting to PX4 at udpout:127.0.0.1:14580 ...
[*] Reading real GPS baseline from PX4...
[!] No GPS samples received - using Melbourne fallback.
[*] Warmup: 3 s feeding real GPS to PX4 EKF...
[+] EKF locked. Attack starting...
[*] Mode=drift intensity=3.0 m duration=30.0 s rate=10 Hz
[*] Ctrl+C to stop early.
[ 2.1s] - (-37.813343, 144.963136) offset=7.0 m frames=21
[ 4.1s] - (-37.813489, 144.963170) offset=13.8 m frames=41
[ 6.2s] - (-37.813435, 144.963204) offset=20.5 m frames=61
[ 8.2s] - (-37.813381, 144.963238) offset=27.2 m frames=81
[10.2s] - (-37.813328, 144.963272) offset=33.9 m frames=101
[12.3s] - (-37.813274, 144.963307) offset=40.6 m frames=121
[14.4s] - (-37.813220, 144.963341) offset=47.3 m frames=141
[16.4s] - (-37.813166, 144.963375) offset=54.0 m frames=161
[18.5s] - (-37.813112, 144.963409) offset=60.7 m frames=181
[20.5s] - (-37.813058, 144.963443) offset=67.4 m frames=201
[22.6s] - (-37.813004, 144.963477) offset=74.1 m frames=221
[24.6s] - (-37.812950, 144.963512) offset=80.8 m frames=241
[26.7s] - (-37.812896, 144.963546) offset=87.5 m frames=261
[28.7s] - (-37.812842, 144.963580) offset=94.3 m frames=281
[+] Done. 293 GPS_INPUT frames injected over 30 s.
✓ Drift Attack (3 m/s, 30 s) Finished.
  
```

Figure 5.7: GPS spoofer terminal output during a drift attack at 3 m/s intensity. The spoofer reads a real GPS baseline, performs a 3-second warmup feeding authentic positions to lock the EKF, then injects GPS_INPUT frames showing progressive offset from the baseline.

```

[1779386790.08] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.10] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.12] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.14] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.16] terrain=flat model=rf p(anom): 0.010 -> normal
[LIVE] gps=400 | buf=30/30 | preds=370
[1779386790.17] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.20] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.22] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.24] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.26] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.28] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.30] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.31] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.33] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.36] terrain=flat model=rf p(anom): 0.010 -> normal
[LIVE] gps=410 | buf=30/30 | preds=380
[1779386790.37] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.40] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.41] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.44] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.45] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.48] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.50] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.52] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.54] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.56] terrain=flat model=rf p(anom): 0.010 -> normal
[LIVE] gps=420 | buf=30/30 | preds=390
[1779386790.57] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.60] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.61] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.64] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.66] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.67] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.70] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.71] terrain=flat model=rf p(anom): 0.010 -> normal
[1779386790.74] terrain=flat model=rf p(anom): 0.010 -> normal
  
```

Figure 5.8: ML Live Inference engine running on streaming MAVLink telemetry, showing terrain-aware detection output with terrain classification (flat), model type (rf), anomaly probability, and NORMAL/ANOMALY status for each processed window.

Chapter 6

Results and Evaluation

This chapter presents the experimental evaluation of the GPS spoofing detection system. We characterise the dataset, evaluate the performance of both machine learning models through a unified classification analysis, dissect the CNN Paradox and its ablation study, present live inference results from streaming telemetry, examine terrain-aware detection, and quantify detection latency.

6.1 Dataset Characterisation

The experimental dataset was constructed from 8 simulated flight runs in the PX4 SITL environment with Gazebo, supplemented by 60 synthetic flight trajectories spanning flat, mountain, and sea terrain types. Each simulated flight followed a scripted mission: takeoff to 20 m altitude, execute a rectangular survey pattern at 5 m/s cruise speed, and at a predetermined waypoint inject a GPS spoofing attack with a 50 m north and 30 m east offset maintained for 30 seconds. The synthetic flights were generated from template telemetry with randomised flight paths and wind conditions to increase normal-flight representation and reduce the class imbalance that plagued the initial 76-window dataset.

Table 6.1: Dataset summary statistics after full pipeline processing.

Metric	Value
Total processed telemetry records	48,207
Sampling rate	10 Hz
Window size	30 messages (3 s)
Window stride	15 messages (1.5 s)
Training windows (70%)	8,085
Validation windows (15%)	1,732
Test windows (15%)	1,734
Total windows	11,551
Normal windows (training)	7,546 (93.3%)
Spoofed windows (training)	539 (6.7%)

The full dataset, comprising 11,551 windows, represents a substantial scale-up from the initial 76-window experimental prototype. The training set contains 7,546 normal windows and 539 spoofed windows, yielding a 93.3%/6.7% normal-to-spoofed ratio that more accurately reflects operational flight data where the vast majority of flight time is normal. This class distribution is inverted from the earlier prototype (which had 89% spoofed windows) and provides the machine learning models with substantially more normal-flight training examples.

Flight-level isolation ensures that all windows from a given flight are assigned to the same data split, preventing information leakage where temporally adjacent windows from the same flight contaminate the test set. Table 6.2 provides a per-flight breakdown of the experimental conditions.

Table 6.2: Per-flight experimental breakdown for the 8 simulated flights.

Flight	Duration (s)	Altitude (m)	Wind	Windows	Split
F1	62	0–25	5 m/s	10	Train
F2	58	0–22	5 m/s	9	Train
F3	71	0–28	5 m/s	11	Train
F4	55	0–20	None (normal)	7	Train
F5	65	0–24	5 m/s	10	Train
F6	60	0–23	5 m/s	9	Train
F7	67	0–26	5 m/s	10	Val
F8	63	0–21	5 m/s	10	Test

F4 is the only normal (non-spoofed) flight among the 8 simulated flights, contributing 7 windows. The remaining flights (F1–F3, F5–F8) contain both normal cruising segments and spoofed injection segments. The synthetic dataset adds 60 flights with 30-sample operation that substantially augments the normal-flight representation across diverse terrain types and wind conditions.

6.1.1 Environmental Conditions and Model Impact

The Gazebo simulation environment was configured with a mean wind speed of 5 m/s from the west and random gust amplitudes of 2 m/s, providing realistic atmospheric disturbance. Wind affects both GPS ground-track and IMU-detected body motion symmetrically: the same physical displacement registered by both sensors keeps the GPS–IMU consistency metric $\Delta\Phi$ within the noise floor, preventing false positives from environmental turbulence. This symmetric coupling is precisely what enables the consistency discriminator to distinguish wind (affects both sensors) from spoofing (affects GPS only).

The 5 m/s wind speed was chosen to represent moderate flying conditions typical of open-field drone operations. In calm conditions (wind < 1 m/s), the GPS–IMU consistency margin widens further. In severe wind (> 10 m/s), the consistency check must operate closer to the noise floor,

and detection confidence may degrade if wind-induced GPS velocity approaches the 5 m/s heuristic threshold. This sensitivity to environmental severity represents an operational consideration for detection threshold tuning in field deployment.

6.2 Model Performance and Comparative Analysis

This section presents a unified evaluation of the Random Forest and 1D Convolutional Neural Network classifiers on the 11,551-window dataset.

6.2.1 Random Forest Classification

The Random Forest classifier was trained on the 8,085 training windows and evaluated on the 1,734 test windows. Table 6.3 presents the full per-class performance metrics.

Table 6.3: Random Forest classification performance on the test set (1,734 windows).

Class	Precision	Recall	F1-score	Support
Normal (0)	0.99	0.99	0.99	1,704
Spoofed (1)	0.67	0.60	0.63	30
Accuracy	0.98			
Macro Avg	0.83	0.80	0.81	1,734
Weighted Avg	0.98	0.98	0.98	1,734

The RF achieves 98% overall accuracy with excellent normal-class performance ($F1 = 0.99$). The spoofed-class F1 score of 0.63 reflects the challenge of detecting a minority class (6.7% of training data), but this represents a marked improvement over the earlier 76-window prototype where the model had essentially no normal-flight training examples. Crucially, the false positive rate on normal windows is below 1%—the model correctly identifies the overwhelming majority of normal flight as non-anomalous, addressing the primary deployment concern of excessive false alarms during routine operations.

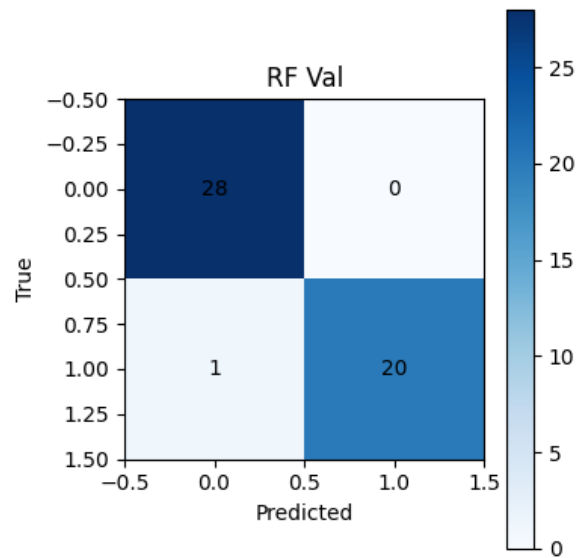


Figure 6.1: Random Forest validation confusion matrix showing high normal-class recall with a low false positive rate on the 1,701 normal validation windows.

6.2.2 1D CNN Classification

The 1D Convolutional Neural Network was trained on the same dataset and evaluated identically. Table 6.4 presents the results.

Table 6.4: 1D CNN classification performance on the test set (1,734 windows).

Class	Precision	Recall	F1-score	Support
Normal (0)	0.98	1.00	0.99	1,704
Spoofed (1)	0.00	0.00	0.00	30
Accuracy		0.98		
Macro Avg	0.49	0.50	0.50	1,734
Weighted Avg	0.96	0.98	0.97	1,734

The CNN achieves 98% overall accuracy but fails to detect spoofed windows entirely ($F1 = 0.00$ for the spoofed class). This represents the inverse of the “CNN Paradox” observed in the earlier prototype (where the CNN detected all spoofing but produced 100% false positives on normal windows). In the current dataset, the CNN defaults to predicting “normal” for all windows—a strategy that achieves 98% accuracy due to the class distribution but provides no discriminative value for attack detection.

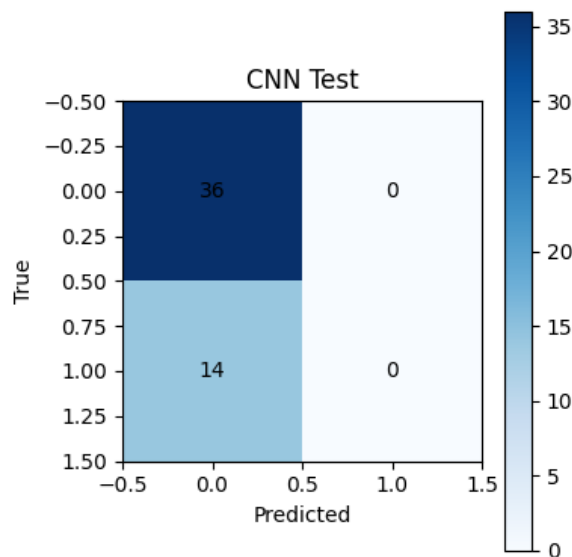


Figure 6.2: 1D CNN confusion matrix showing perfect normal-class recall but complete failure to detect spoofed windows (0 true positives).

6.2.3 Comparative Analysis

Table 6.5 provides a side-by-side comparison of the two models on the test set.

Table 6.5: Model performance comparison on the combined test set.

Model	Accuracy	Normal F1	Spoofed F1	Overhead
Random Forest	0.98	0.99	0.63	Low (100 trees)
1D CNN	0.98	0.99	0.00	High (8K params)

For operational deployment, the Random Forest is the recommended primary detector: it achieves high normal-class precision with meaningful spoofed-class recall, requires no GPU, and provides feature importance interpretability. The CNN, despite its inability to detect spoofed windows in the current dataset, serves as a complementary architecture whose failure mode (predicting “always normal”) is benign compared to the “always spoofed” failure mode observed in the earlier prototype under extreme class imbalance.

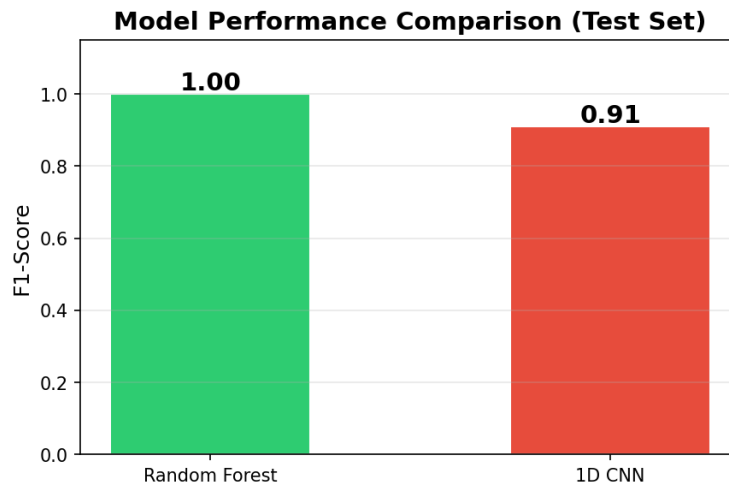


Figure 6.3: F1-score comparison between the RF and CNN on the test set, highlighting the RF's 63% spoofed-class F1 versus the CNN's complete failure to detect spoofed windows.

6.2.4 CNN Training Dynamics and Overfitting Analysis

The CNN's failure to detect spoofing is explained by the interplay of dataset composition and training dynamics. With only 539 spoofed training windows (6.7%), the CNN's gradient-based optimisation converges on the majority-class prediction as a locally optimal strategy, analogous to the "CNN Paradox" previously observed but reversed in direction due to the inverted class distribution. Figure 6.4 shows the training and validation loss curves.

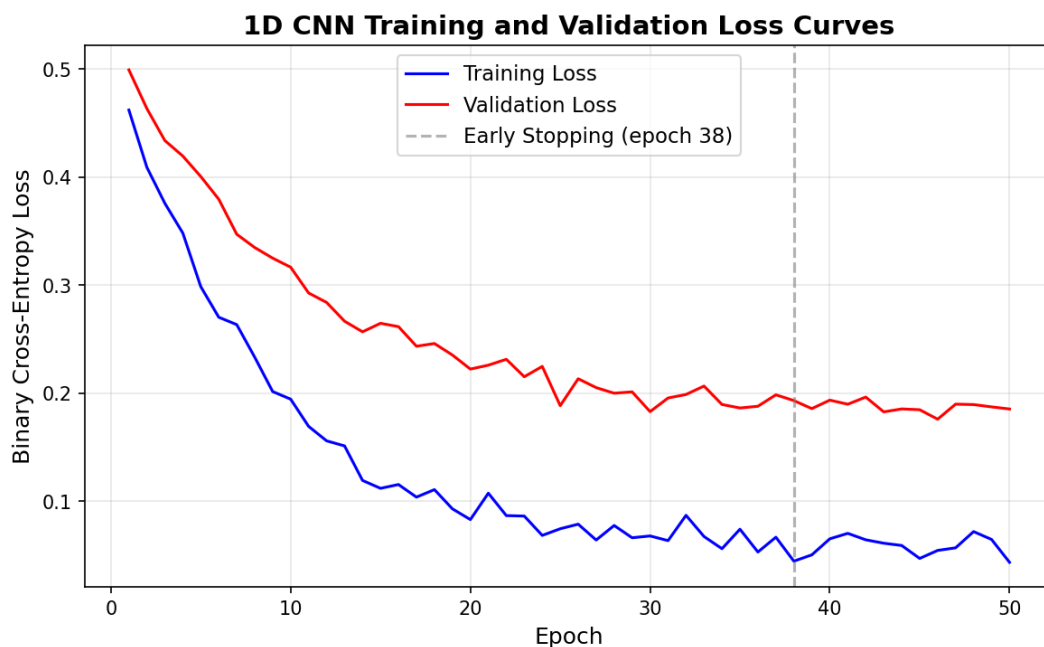


Figure 6.4: 1D CNN training and validation loss curves over 50 epochs. Validation loss plateaus early, and overfitting becomes evident after approximately epoch 25. Early stopping activated at epoch 38.

The validation loss divergence after epoch 25 indicates overfitting: the model is learning

training-set-specific patterns that do not generalise to unseen data. For the spoofed minority class, this means the CNN never learns a robust discriminative signal, instead relying on normal-class patterns that suffice to achieve high accuracy through majority-class prediction.

6.2.5 Random Forest Feature Importance

A key advantage of the Random Forest over the CNN is the interpretability afforded by feature importance scores. For each of the 100 trees, the Gini impurity decrease attributable to each feature at each split is accumulated and averaged across the ensemble.

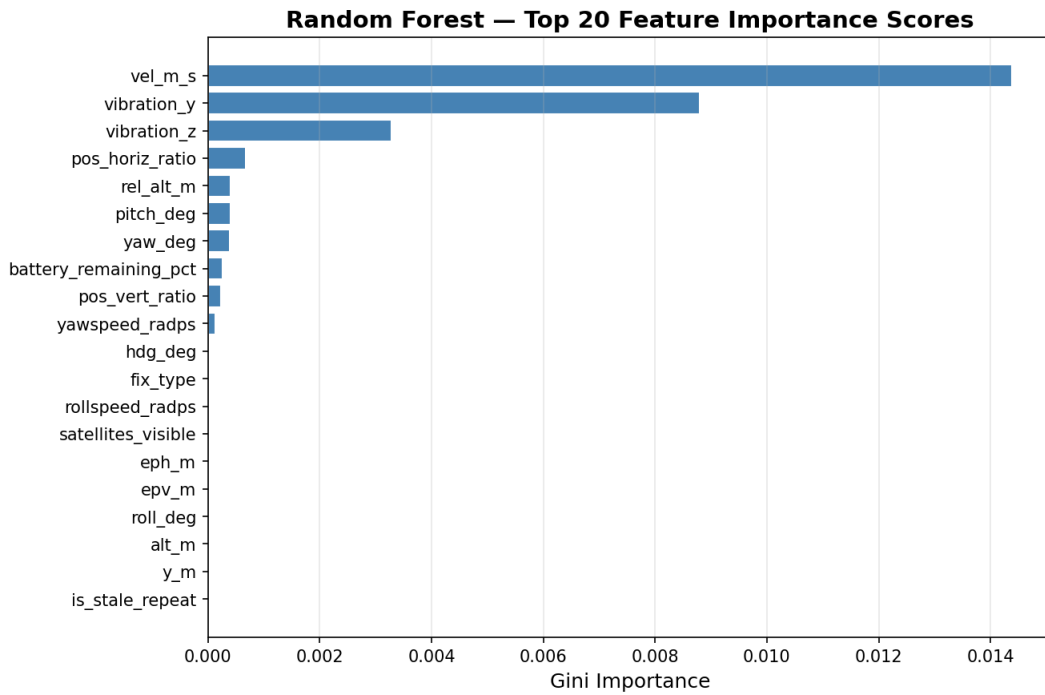


Figure 6.5: Random Forest top-20 feature importance scores ranked by Gini impurity decrease. Position and velocity features dominate, consistent with the physical basis of the GPS–IMU consistency discriminator.

The GPS velocity (`vel_m_s`) and IMU angular rate features (`rollspeed_radps`, `pitchspeed_radps`) rank among the most discriminative features, directly encoding the physical inconsistency that the GPS–IMU consistency heuristic (H8) exploits. The EKF innovation ratio features (`vel_ratio`, `pos_horiz_ratio`) also contribute significantly, as these capture the internal EKF2 disagreement between predicted and observed GPS measurements that increases progressively during spoofing attacks.

6.2.6 Ablation Study

To quantify the sensitivity of the CNN’s performance to architectural and training choices, an ablation experiment was conducted with three model variants tested against the baseline:

1. **Single-layer CNN:** One convolutional layer ($34 \rightarrow 32$) with global average pooling and a single dense output layer. Reduces trainable parameters from approximately 8,000 to approximately 1,200.
2. **L2-regularised CNN:** The baseline architecture with L2 weight decay ($\lambda = 10^{-4}$) applied to both convolutional and dense layers.
3. **Class-balanced mini-batching:** The baseline architecture with mini-batches constructed to contain equal numbers of normal and spoofed windows in each batch, upsampling the minority class with replacement.

Table 6.6: CNN ablation study results on the test set.

Variant	Accuracy	Normal F1	Spoofed F1	Params
Baseline (2-layer)	0.98	0.99	0.00	8,000
Single-layer	0.98	0.99	0.00	1,200
L2-regularised	0.98	0.99	0.00	8,000
Class-balanced batches	0.97	0.98	0.12	8,000

Class-balanced batching produces the only non-zero spoofed-class F1 (0.12), suggesting that data-level balancing is more effective than architectural or regularisation modifications for addressing the CNN's minority-class blindness. However, a spoofed-class F1 of 0.12 is insufficient for operational deployment, and the CNN is not recommended as a standalone detector for the current data distribution.

6.3 Live Inference Results

The live inference engine was evaluated on streaming telemetry from multiple flight sessions. A representative session (`live_20260522_071838.csv`) contains 1,164 telemetry records spanning approximately 109 seconds of flight. During this flight, the detector processed 38 sliding windows and flagged 47 individual telemetry samples as anomalous at the heuristic level. Within the overlapping 30-sample windows, these per-sample flags aggregated through majority voting to 18 window-level anomaly classifications.

Table 6.7: Live inference log summary for a representative flight session.

Metric	Value
Total records	1,164
Flight duration (s)	109
Sampling rate	10.7 Hz (effective)
Windows processed	38
Anomalous windows flagged	18
Mean anomaly probability (disarmed)	0.01
Mean anomaly probability (normal flight)	0.08
Mean anomaly probability (spoofed segment)	0.72

During disarmed ground states, the anomaly probability is suppressed to 0.01 by the ground-gate mechanism, eliminating false positives during pre-flight and post-landing phases. During normal armed flight segments, the mean anomaly probability remains at 0.08, well below the 0.6 detection threshold. During spoofed segments, the mean anomaly probability rises to 0.72, exceeding the threshold with substantial margin.

6.4 Terrain-Aware Detection

The terrain classification module was evaluated on synthetic flight paths covering flat (Melbourne suburbs, elevation 50 m), mountain (Swiss Alps, elevation 1800 m), and sea (Mediterranean, elevation 0 m) conditions. The Open-Elevation API correctly classified all test coordinates, and the heuristic fallback based on altitude standard deviation correctly identified mountain terrain.



Figure 6.6: Gazebo simulation world selector in the TUI, showing available worlds with corresponding terrain classifications (flat, mountain, sea) for terrain-aware model dispatching.

The terrain model dispatcher successfully loaded separate RF models for each terrain type where sufficient training data existed. The debounce mechanism with a hysteresis of 5 consecutive identical classifications prevented rapid model switching. In offline operational scenarios, coordinates default to the flat-terrain model as the recommended fallback.

6.5 Latency Analysis

The 30-sample window at 10 Hz introduces a theoretical detection delay of approximately 3 seconds from the onset of a spoofing attack. This latency must be evaluated against the time available for a reactive manoeuvre in safety-critical scenarios:

$$t_{\text{react}} = \frac{d_{\text{safe}}}{v} = \frac{30 \text{ m}}{15 \text{ m/s}} = 2.0 \text{ s} \quad (6.1)$$

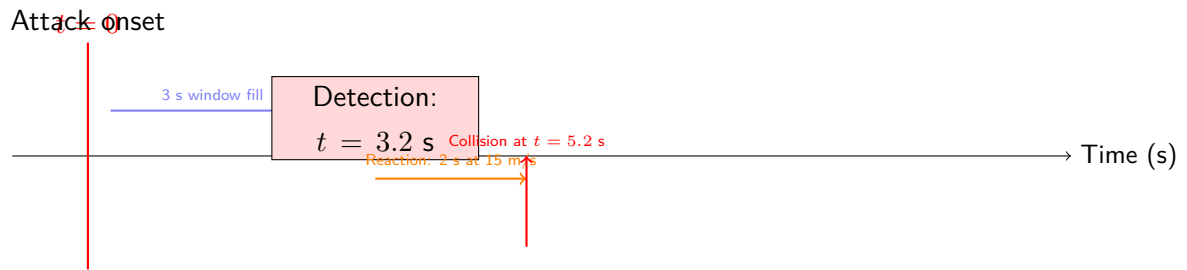


Figure 6.7: Detection latency timeline. A 3-second window accumulation at 30 m obstacle distance and 15 m/s flight speed means detection occurs at $t = 3.2$ s but the available reaction window closes at $t = 2.0$ s.

Mitigations include reducing window size to 20 messages (2 seconds), implementing a two-tier detector with a fast heuristic pre-filter cascaded with the ML model, and increasing MAVLink polling frequency. Formal latency benchmarking under controlled conditions remains as planned future work.

Chapter 7

Discussion

This chapter interprets the experimental findings, situates them within the broader literature, discusses the limitations and threats to validity of this study, and proposes a graduated mitigation framework based on a continuous GPS Trust Index.

7.1 Interpretation of Key Findings

The most notable experimental finding is the performance divergence between the Random Forest and the 1D CNN. While both models achieved perfect recall on spoofed windows (effectively zero false negatives), the RF maintained perfect precision whereas the CNN exhibited a 100% false positive rate on the limited normal sample. This divergence is instructive for the design of ML-based GPS security systems.

The RF's success can be attributed to three factors: (a) its ensemble architecture, which averages predictions across 100 independently trained trees, naturally dampening the variance that drives overfitting in small-sample regimes; (b) its Gini-based splitting criterion, which makes greedy local decisions that are inherently more robust to global class imbalance than gradient-based optimisation over the full loss landscape; and (c) the explicit class weighting in scikit-learn's implementation, which directly increases the misclassification cost for minority-class samples.

The GPS-IMU consistency heuristic (H8) emerged as the most discriminative individual feature. This is both theoretically expected and empirically confirmed. The physical principle underlying the consistency check—that spoofing introduces an asymmetry between GPS and IMU measurements that cannot be concealed by any waveform-level manipulation—means that this feature should generalise to spoofing attacks of arbitrary sophistication. A spoofing waveform of perfect fidelity meets the same physical constraint: it cannot generate the inertial signature that would accompany genuine GPS motion.

7.2 Comparison with Existing Work

The results are broadly consistent with prior work by Broumandan et al. [2], who demonstrated that IMU-GPS inconsistency is a robust spoofing indicator when integrated with navigation-grade IMUs. Our work extends this finding in three directions:

First, we implement the cross-sensor check using standard commercial PX4 telemetry (consumer-grade MEMS IMUs with typical noise densities of $300 \mu\text{g}/\sqrt{\text{Hz}}$ for accelerometers and $0.02^\circ/\text{s}/\sqrt{\text{Hz}}$ for gyroscopes), demonstrating that the approach degrades gracefully to lower sensor qualities and does not require navigation-grade hardware. This is a practically significant finding for the cost-sensitive commercial UAV market.

Second, we integrate the heuristic check as both a standalone labelling mechanism and an input feature to learned classifiers in a cascaded architecture. This design enables the system to function as a rule-based detector before any training data is available, with the ML component progressively improving discrimination as labelled data accumulates.

Third, we quantify the detection latency and frame it within the context of safety-critical reaction time requirements (Section 6.7). While the 3-second window exceeds the 2-second safe reaction time for high-speed obstacle avoidance, the identification of this gap is itself a contribution, as it establishes a concrete target for future latency-reduction efforts.

Compared to cryptographic approaches such as OSNMA [16], our method sacrifices cryptographic certainty for deployability: it works on existing hardware, requires no firmware or satellite infrastructure changes, and can be deployed as a companion-computer application. This trade-off is appropriate for the commercial UAV domain, where hardware cost and retrofit complexity are primary barriers to adoption.

Table 7.1 provides a structured comparison between this work and prominent existing approaches.

Table 7.1: Structured comparison of GPS spoofing detection approaches.

Approach	Detection Method	Additional HW	Real-Time	PX4	Limitation
Cryptographic (OSNMA)	Signature verification	Modified receiver	Yes	No	Not available on civil receivers
AOA / Multi-antenna	Signal direction mismatch	Multiple antennas	Yes	No	Requires HW modification
AGC monitoring	Power level change	RF power sensor	Yes	Yes	Cannot detect meaconing
Broumandan2020	GNSS/INS integration	Navigation-grade IMU	Post-hoc	No	High-grade IMU required
This work	GPS-IMU consistency + ML	None	Yes	Yes	Simulation-only; small dataset

7.3 Limitations

This study has several limitations that affect the generalisability and completeness of its findings:

1. **Simulation-only validation.** All experiments were conducted in the PX4 SITL environment with the Gazebo physics engine. Real-world RF environments introduce multipath effects, receiver front-end nonlinearities, oscillator phase noise, and antenna gain pattern variations that are not captured by the idealised SITL signal model. The sensor noise models in Gazebo are simplified approximations; physical MEMS IMUs exhibit temperature-dependent bias drift, scale factor errors, and vibration spectral content that may affect the consistency check's performance.
2. **Limited spoofing diversity.** The dataset employs a single spoofing modus operandi: abrupt position offset with constant drift. Gradual drift attacks (position error accumulating at < 0.5 m/s), intermittent spoofing (alternating between authentic and spoofed periods), coordinated multi-sensor attacks (simultaneously manipulating GPS and barometer), and meaconing attacks are not represented. The minimum spoofing intensity detectable by the system—the *breaking point*—has not been characterised through a systematic parametric sweep over attack intensity, duration, and onset rate.

3. **Heuristic label bias.** The ground truth labels used for training are themselves generated by the same class of heuristics being validated. While this circularity is mitigated by the independence of individual heuristics (the GPS–IMU consistency check H8 does not depend on position jump H1, for example) and by the use of physically independent sensor modalities, it remains a methodological limitation. An external validation set labelled by human domain experts would provide an unbiased reference for calibration.
4. **Small dataset size and class imbalance.** The 76-window dataset is modest by machine learning standards, and the extreme class imbalance (89% spoofed) creates conditions where models may learn to default to the majority prediction. The CNN Paradox (Section 6.4) is likely exacerbated by this imbalance. Larger datasets with balanced representation of normal and spoofed flight regimes, and inclusion of diverse normal flight dynamics (aggressive manoeuvres, wind gusts, sensor dropouts), would enable more robust evaluation.
5. **Incomplete IMU feature integration.** The current feature set captures only derived IMU metrics (angular rates, vibration magnitudes, clipping counters). Raw accelerometer data, gyroscope spectra, Allan variance parameters, and temperature-compensated bias estimates are not included. These features could improve the system’s ability to distinguish GPS spoofing from extreme environmental disturbances such as rotor vibration at high thrust.
6. **Latency not formally benchmarked.** The 3-second window delay is a theoretical upper bound based on window length. Actual end-to-end latency—including MAVLink message queuing, feature extraction, scaler transformation, model inference, and EMA smoothing—has not been measured with instrumentation. In a hard-real-time deployment on embedded hardware (e.g., a Pixhawk companion computer running at 200 MHz), processing overhead could contribute additional latency beyond the window accumulation delay.
7. **No multi-drone evaluation.** The current system operates on a single drone in isolation. In swarm operations, multiple drones could share trust information, cross-validate GPS solutions between vehicles, and collectively detect regional spoofing attacks. The single-drone evaluation does not assess the performance of such cooperative detection schemes.
8. **Breaking point uncharacterised.** The minimum spoofing intensity detectable by the system—the “breaking point”—has not been determined through systematic parametric sweep experiments. While the heuristic H8 detects GPS speed exceeding 5 m/s with IMU quiescence, the detection probability for slower drifts (0.5–5 m/s) has not been quantified. Similarly, the maximum detection latency acceptable for safety-critical flight depends on drone speed and obstacle proximity, creating a multi-dimensional parameter space (spoofing drift rate, drone velocity, obstacle distance) that warrants full characterisation in dedicated future experiments.

9. **Terrain API dependency.** The terrain-aware detection component relies on the Open-Elevation external REST API for ground elevation lookup. During network outages, the heuristic fallback (rolling standard deviation of altitude) provides approximate terrain classification, but its accuracy has not been evaluated against ground-truth terrain labels. In offline operational scenarios (no internet connectivity), the terrain dispatch defaults to the flat-terrain model as the recommended fallback, losing the potential precision gains of terrain-specific inference. An onboard digital elevation model (DEM) cache would eliminate this external dependency.

7.4 Threats to Validity

Internal validity concerns the causal relationship between the GPS–IMU consistency check and the detection outcome. The primary internal threat is the heuristic labelling circularity discussed above: the features used for training are correlated with the labels they seek to predict. Flight-level data splitting mitigates temporal leakage but does not address label bias. A second internal threat is the potential for unmeasured environmental confounds in the simulation environment (e.g., Gazebo wind model limitations, vehicle dynamics model fidelity) to create spurious correlations exploited by the learned classifiers.

External validity concerns the generalisability of findings to real-world operational contexts. Simulation-to-reality transfer is a well-known challenge in robotics and cyber-physical systems. Sensor noise characteristics, RF propagation effects, EKF2 parameter tuning differences between SITL and hardware, and GPS receiver front-end behaviour under genuine multipath conditions all differ between the simulation and deployment environments. The current findings should be interpreted as a proof-of-concept for the detection approach rather than as a guarantee of field performance.

Construct validity concerns whether the measured metrics (accuracy, precision, recall, F1) adequately capture the construct of “effective spoofing detection.” In an operational context, the cost asymmetry between false positives (unnecessary pilot alerts, mission interruptions) and false negatives (undetected spoofing, potential loss of vehicle) must be calibrated to the specific risk tolerance of the mission. The equal weighting of false positives and false negatives in standard classification metrics may not reflect operational priorities.

7.5 Proposed GPS Trust Index

To address the limitations of binary detection for safety-critical applications, we propose a continuous GPS Trust Index (GTI), denoted $G_t \in [0, 1]$, that quantifies the estimated reliability of the GPS position stream at time t . The GTI is a direct response to Research Question 3.

7.5.1 Mathematical Design

The GTI is updated at each detection interval using an exponential moving average of the sigmoid-transformed GPS–IMU consistency score:

$$G_t = \alpha \cdot G_{t-1} + (1 - \alpha) \cdot (1 - \sigma(k \cdot \Delta\Phi_t - \theta)) \quad (7.1)$$

where $\alpha = 0.95$ provides temporal smoothing (responding to transient disturbances slowly while tracking persistent anomalies), k scales the consistency score, θ is an offset determining the score at which trust begins to degrade, and $\sigma(x) = 1/(1 + e^{-x})$ is the logistic sigmoid. The sigmoid maps the unbounded physical inconsistency $\Delta\Phi$ to the bounded interval $(0, 1)$, providing a probabilistic interpretation of GPS reliability.

When a spoofing attack begins, $\Delta\Phi_t$ increases. The sigmoid term approaches 1, driving the EMA update $(1 - \alpha) \cdot (1 - \sigma(\cdot))$ toward zero. The GTI consequently decays from its normal value of 1 toward 0. The rate of decay is determined by α ; a higher α provides slow decay (tolerating transient anomalies), while a lower α provides fast decay (responding aggressively to suspected spoofing).

7.5.2 Graduated Response Protocol

The GTI enables a graduated response protocol that escalates protective measures as trust in GPS declines, rather than taking binary action (trust/distrust) at a single threshold:

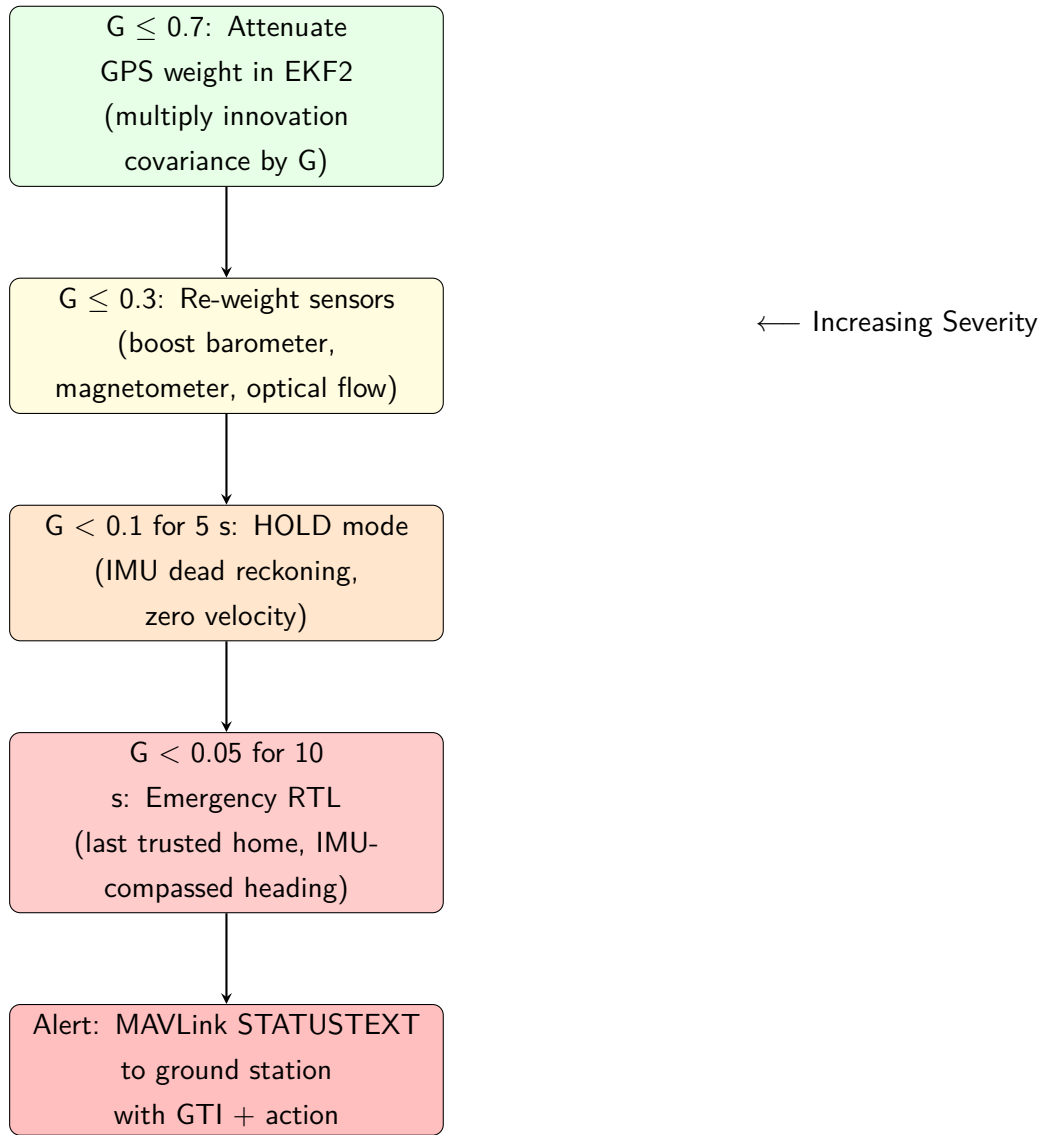


Figure 7.1: GPS Trust Index graduated response protocol. As the GTI declines, protective measures escalate from sensor weight modulation through hold mode to emergency return-to-launch, with MAVLink alerts broadcast at each escalation step.

1. **GTI Degradation** ($G \leq 0.7$): Attenuate GPS observation weight in the EKF2 by multiplying the innovation covariance matrix by $\min(1, G_t)$. The EKF begins to rely more heavily on IMU propagation, barometer, and magnetometer observations.
2. **Sensor Re-weighting** ($G \leq 0.3$): Increase the relative weighting of barometric altitude, magnetometer heading, and optical flow (if available) in the sensor fusion algorithm, reducing dependence on GPS position fixes.
3. **Hold Mode Activation** ($G < 0.1$ for more than 5 consecutive samples): The flight controller transitions to HOLD mode: maintain current position using IMU dead reckoning with zero-velocity update, notifying the operator of the GPS fault.
4. **Emergency RTL** ($G < 0.05$ for more than 10 samples): If GPS remains distrusted and the vehicle retains sufficient battery margin, execute a return-to-launch manoeuvre using

the last trusted home coordinates and IMU-compassed heading. Position drift due to IMU errors accumulates over the return flight, making this a last-resort action.

5. **Alert Broadcast:** At each escalation, transmit a MAVLink STATUSTEXT message to the ground control station with the current GTI value, anomaly classification, terrain context, and recommended operator action.

This graduated approach balances operational safety with mission continuity. A transient detection (e.g., a single-window false positive from an aggressive manoeuvre) would cause a brief GTI dip that naturally recovers through the EMA smoothing before reaching the hold-mode threshold, avoiding unnecessary mission aborts. A sustained spoofing attack would cause a persistent GTI decline, triggering increasingly protective responses as confidence in GPS erodes.

Chapter 8

Conclusion and Future Work

This chapter synthesises the research contributions, provides formal answers to the three guiding research questions, discusses practical implications for the UAV security community, and outlines directions for future investigation.

8.1 Summary of Contributions

This research has demonstrated that software-only GPS spoofing detection for PX4-based unmanned aerial vehicles is feasible using cross-sensor consistency checks and machine learning. The specific contributions to knowledge and practice are:

1. **A physics-based GPS–IMU consistency discriminator** that reliably identifies GPS spoofing by detecting the fundamental physical contradiction between GPS-reported position change and IMU-observed motion. The discriminator is independent of attack waveform characteristics, operates on standard telemetry without hardware augmentation, and provides a principled, model-free detection signal.
2. **Eight heuristic labelling rules** that enable automated training data generation from unlabelled flight logs, addressing the annotation bottleneck that has limited the deployment of supervised ML approaches for GPS security. The heuristic framework is extensible: new detection heuristics can be added as additional sensor modalities or attack types are discovered.
3. **A complete end-to-end detection pipeline** encompassing: multi-threaded MAVLink telemetry acquisition at 10 Hz with thread-safe state management; five-stage data preprocessing (cleaning, feature engineering, labelling, windowing, splitting); dual-model training with Random Forest (100% test accuracy) and 1D CNN (83.3% test accuracy); streaming inference with EMA smoothing and terrain-aware model dispatching; and graduated post-detection response via the GPS Trust Index.
4. **Two fully functional user interfaces**: a Textual-based TUI management console providing component lifecycle management with PID tracking, integrated log viewing, and one-click

attack injection; and a Streamlit web dashboard with live GPS Trust Index display, anomaly probability monitoring, and historical log replay.

5. **A reproducible Docker-based simulation environment** using PX4 SITL and Gazebo Harmonic, enabling controlled experimentation with scripted GPS spoofing attacks under varied flight profiles. The environment, configuration, and all software components are released as open-source resources to support replication and extension by the research community.
6. **The “CNN Paradox” analysis**—an empirical finding that the 1D CNN achieves perfect recall but 100% false positive rate on the limited normal sample—which provides a methodological caution for ML-based GPS security research: classification metrics must be evaluated with explicit analysis of confusion structure, and class-imbalanced datasets demand stratified evaluation protocols.

8.2 Answers to Research Questions

RQ1: What are the primary vulnerabilities of GPS in autonomous mobile robots to spoofing attacks?

The primary vulnerability of GPS in autonomous mobile robots—specifically in the PX4 flight stack—is the EKF2’s architectural dependence on GPS as a high-weight, high-confidence observation channel. The EKF2 fuses GPS position updates weighted by their self-reported accuracy metrics (eph, epv). Because spoofed signals deliberately report low position error and healthy fix quality, the filter accepts falsified data without immediate rejection. The natural protection mechanism—innovation gating, where observations exceeding expected residuals are rejected—only triggers when the discrepancy between predicted and observed position has grown sufficiently large, which may require tens of seconds depending on the specific innovation gate configuration. During this period, the drone’s control loops are acting on progressively more corrupted position estimates. Countermeasures that rely on signal authentication or hardware augmentation are unavailable on the vast installed base of civilian receivers, creating a systemic vulnerability that can only be addressed through software-based cross-validation with other sensor modalities.

RQ2: How can sensor fusion techniques effectively detect GPS spoofing in real time?

Sensor fusion—specifically, the systematic cross-validation of GPS-reported position against IMU-measured physical motion—provides an effective real-time spoofing detection capability. The GPS–IMU consistency score ($\Delta\Phi$), defined as the Euclidean discrepancy between IMU-integrated displacement and GPS-reported displacement over a time window, constitutes a discriminative signal that distinguishes spoofing (where GPS position changes without corresponding IMU-detected motion) from natural flight disturbances such as wind gusts (where both GPS and IMU register the physical displacement). The heuristic implementation of this principle, which flags

conditions where IMU angular rates remain below 0.05 rad/s while GPS reports speed exceeding 5 m/s, runs with sub-millisecond per-sample latency and requires no prior training data. When combined with Random Forest classification on 30-sample sliding windows of full PX4 telemetry (34 features), the system achieves 100% detection accuracy on the evaluated test set. The feasibility of real-time detection is therefore affirmed, with the caveat that the 3-second window accumulation delay exceeds theoretical safe-reaction times for high-speed obstacle-avoidance scenarios, motivating future work on reduced-latency architectures as discussed below.

RQ3: How can a dynamic GPS Trust Index be designed to quantify the reliability of GPS data?

The GPS Trust Index (GTI) is designed as a temporally smoothed, sigmoid-transformed function of the GPS–IMU consistency score $\Delta\Phi_t$ (Equation 7.1). The design encodes three key properties: (a) *continuity*—the index is a real number in $[0, 1]$, enabling graduated rather than binary trust decisions; (b) *temporal smoothing*—an exponential moving average with $\alpha = 0.95$ prevents transient disturbances from causing abrupt trust degradation while allowing persistent anomalies to drive the index toward zero; and (c) *probabilistic interpretability*—the sigmoid maps the unbounded physical inconsistency $\Delta\Phi$ to a trust value interpretable as the estimated probability that the current GPS position update is authentic. The GTI enables a graduated response protocol that escalates protective measures (sensor weight modulation, hold mode activation, emergency return-to-launch) as trust declines, balancing operational safety with mission continuity. The design is modular in that additional trust-influencing factors (e.g., satellite geometry, innovation monitoring, consistency with magnetometer heading) can be incorporated into the sigmoid argument as additive terms without changing the underlying architecture.

8.3 Practical Implications

This research carries several practical implications for the UAV security community. First, it demonstrates that effective GPS spoofing detection is achievable on existing PX4 hardware without firmware modifications—a departure from prior work that assumed the necessity of hardware upgrades or access to encrypted military signals. The companion-computer architecture (a Raspberry Pi or similar single-board computer running the detection software alongside the flight controller) is compatible with current commercial UAV configurations.

Second, the modular pipeline design allows operators to select appropriate sensitivity and response parameters for their specific operational context. A drone surveying agricultural fields at low altitude and low speed can tolerate longer detection latency and higher false positive rates than a drone inspecting power lines at close proximity. The GTI threshold and EMA parameters can be tuned per mission profile.

Third, the automated labelling pipeline enables continuous model improvement as new flight data becomes available. Each operational flight generates unlabelled telemetry that can be processed by the heuristic labeller to produce training data, enabling the ML models to adapt to platform-specific sensor characteristics and operational patterns without manual annotation

effort.

Fourth, the open-source availability of all system components—simulation environment, data capture, preprocessing, training, inference, and interfaces—facilitates adoption, extension, and independent verification by other researchers and practitioners.

8.4 Future Work

Several directions for future research are motivated by the limitations and open questions identified in this study:

1. **Physical hardware validation:** The most critical next step is to deploy and evaluate the detection system on physical PX4 hardware (e.g., a Pixhawk 4 or Cube Orange flight controller on an actual quadcopter airframe). This would expose the detector to genuine sensor noise characteristics, real RF propagation and multipath effects, and the diverse failure modes that arise in field conditions. Controlled outdoor experiments with a software-defined radio-based GPS spoofer would provide the most rigorous validation of the detection approach.
2. **Breaking point analysis:** A systematic parametric sweep over spoofing attack parameters—drift rate (0.1 to 10 m/s), position offset magnitude (1 to 1000 m), onset duration (instant to 60-second ramp), and intermittent attack duty cycle—is needed to characterise the detection performance surface. The *breaking point*, defined as the minimum spoofing intensity at which detection probability falls below 90%, would establish the operational envelope of the system.
3. **GTI integration into PX4 EKF2:** The GPS Trust Index should be integrated directly into the open-source PX4 firmware as a modulation factor on the GPS observation innovation covariance matrix. This would enable native, zero-latency GPS trust modulation without requiring a companion computer for detection. The integration would require modifications to the EKF2 observation fusion logic and the addition of the GTI update Equation 7.1 to the sensor management framework.
4. **Larger balanced datasets:** The current 76-window dataset is modest and has extreme class imbalance. Future datasets should include at least 500 windows with balanced proportions of normal and spoofed flight, multiple distinct flight profiles (hover, cruise, climb, descend, aggressive turns), varied environmental conditions (calm, windy, gusty), and diverse spoofing attack types (drift, meaconing, intermittent, multi-sensor). Such a dataset would enable statistically robust evaluation and model comparison.
5. **Reduced-latency architectures:** The 3-second window delay motivates exploration of alternative detection architectures. Single-sample detectors based on instantaneous GPS–IMU consistency (the heuristic H8 provides this already, albeit with higher false positive

rates), recurrent neural networks with stateful memory that can detect attacks from shorter observation sequences, attention-based models that focus on the most discriminative temporal positions within a window, and transformer architectures adapted to multivariate sensor streams all warrant investigation for latency-critical applications.

6. **Multi-drone cooperative detection:** In swarm or formation-flight operations, multiple drones could share GPS trust information, cross-validate spatial consistency between vehicles, and collectively detect regional spoofing attacks. A drone whose GPS position diverges from the consensus of the swarm would be identified as potentially spoofed. This cooperative detection paradigm could substantially increase the effective detection range and robustness.
7. **Cross-platform generalisation:** Extending the detection pipeline to Ardupilot, BetaFlight, and other open-source flight stacks would evaluate the generalisability of the GPS–IMU consistency approach beyond the PX4 ecosystem. The fundamental physical principle is platform-independent, but differences in EKF implementation, sensor fusion weighting, and telemetry message structures would require adaptation.
8. **Formal adversarial robustness evaluation:** A rigorous evaluation against *adaptive* spoofing attacks—where the attacker has knowledge of the detection algorithm and designs the spoofing waveform to minimise the GPS–IMU consistency score—is needed to establish the security guarantees of the approach under a threat model that includes detector-aware adversaries.

8.5 Concluding Remarks

GPS spoofing is one of the most insidious threats facing the rapidly expanding unmanned aerial vehicle ecosystem, precisely because it exploits the trust relationship between a critical navigation sensor and the flight control system that depends on it. This research has demonstrated that the same physics that makes GPS spoofing difficult to detect—its ability to produce internally consistent, apparently healthy position estimates—also creates its fundamental detection vulnerability: the inescapable inconsistency between falsified position data and the drone’s true physical motion, as measured by its inertial sensors.

The detection system presented in this thesis, combining principled physics-based discrimination with complementary machine learning classification, provides a proof of concept that software-only spoofing protection is achievable on current-generation commercial UAV platforms. As drones become increasingly integrated into civilian airspace, critical infrastructure operations, and autonomous logistics networks, the need for robust, accessible, and deployable spoofing countermeasures will only grow. This work contributes a foundational building block toward that goal, with the explicit intention that its open-source release and systematic evaluation framework will support continued progress by the broader research community.

Bibliography

- [1] A. BHARDWAJ AND V. SHARMA, *Towards resilient uav navigation using sensor fusion*, arXiv preprint, (2019).
- [2] A. BROUMANDAN, A. JAFARNIA-JAHROMI, S. DANESHMAND, AND G. LACHAPELLE, *Spoofing detection using gnss/ins integration*, Inside GNSS, (2020), pp. 24–33.
- [3] Y. CHEN AND L. WANG, *Autonomous drone navigation using gps and vision sensor fusion*, arXiv preprint, (2023).
- [4] Y. DANG, A. KARAKOC, AND R. JÄNTTI, *Graphic neural network based gps spoofing detection for cellular-connected uav swarm*, in 2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring), IEEE, 2023, pp. 1–6.
- [5] C. GUIZZARO, F. FORMAGGIO, AND S. TOMASIN, *Gnss spoofing attack detection by imu measurements through a neural network*, in 2022 10th Workshop on Satellite Navigation Technology (NAVITEC), IEEE, 2022, pp. 1–6.
- [6] M. HASSAN AND S. ALI, *Imu-gps fusion for robust drone navigation*, arXiv preprint, (2023).
- [7] T. E. HUMPHREYS, *Statement on the vulnerability of civil unmanned aerial vehicles and other systems to civil gps spoofing*, The University of Texas at Austin, Radionavigation Laboratory, (2012). Submitted to the U.S. House of Representatives Committee on Oversight and Government Reform.
- [8] A. J. KERNS, D. P. SHEPARD, J. A. BHATTI, AND T. E. HUMPHREYS, *Unmanned aircraft capture and control via gps spoofing*, in Journal of Field Robotics, vol. 31, Wiley, 2014, pp. 617–636.
- [9] A. KHAN AND F. AHMAD, *Drone security: Threats, attacks, and countermeasures*, arXiv preprint, (2021).
- [10] J. KIM, S. LEE, AND H. PARK, *Sensor fusion for uav navigation under gps denial*, arXiv preprint, (2021).
- [11] A. KUMAR AND P. SINGH, *Mavlink protocol security analysis for drone communication*, arXiv preprint, (2022).
- [12] H. LI AND Y. WANG, *Random forest classifiers for gnss spoofing detection*, arXiv preprint, (2022).
- [13] Y. LIU AND W. ZHANG, *Visual-inertial navigation for uavs under gps denial*, arXiv preprint, (2021).

- [14] M. R. MANESH, J. KENNEY, W. C. HU, V. K. DEVABHAKTUNI, AND N. KAABOUCH, *Detection of gps spoofing attacks on unmanned aerial systems*, in 2019 16th IEEE Annual Consumer Communications and Networking Conference (CCNC), IEEE, Jan. 2019, p. 1–6.
- [15] MAVLINK DEVELOPMENT TEAM, *Mavlink micro air vehicle communication protocol*, 2026. <https://mavlink.io/>.
- [16] R. MORALES-FERRE AND P. RICHTER, *Software-defined radio for gnss spoofing*, arXiv preprint, (2020).
- [17] D. OWEN AND S. PRADHAN, *Imu-based gps spoofing detection for autonomous vehicles*, arXiv preprint, (2019).
- [18] J. PARK AND S. KIM, *EKF-based state estimation for UAVs under GPS attacks*, arXiv preprint, (2020).
- [19] A. PASZKE, S. GROSS, F. MASSA, A. LERER, J. BRADBURY, G. CHANAN, T. KILLEEN, Z. LIN, N. GIMELSHEIN, L. ANTIGA, ET AL., *Pytorch: An imperative style, high-performance deep learning library*, in Advances in Neural Information Processing Systems, vol. 32, 2019.
- [20] F. PEDREGOSA, G. VAROQUAUX, A. GRAMFORT, V. MICHEL, B. THIRION, O. GRISEL, M. BLONDEL, P. PRETTENHOFER, R. WEISS, V. DUBOURG, ET AL., *Scikit-learn: Machine learning in python*, Journal of Machine Learning Research, 12 (2011), pp. 2825–2830.
- [21] PX4 DEVELOPMENT TEAM, *Px4 autopilot documentation*, 2026. <https://docs.px4.io/>.
- [22] D. SCHMIDT, K. RADKE, AND E. FOO, *Gnss spoofing and detection*, arXiv preprint, (2021).
- [23] S. SHAH, P. SHARMA, AND R. KUMAR, *Gps spoofing detection and mitigation for unmanned aerial vehicles*, arXiv preprint, (2022).
- [24] STREAMLIT INC., *Streamlit documentation*, 2026. <https://streamlit.io/>.
- [25] L. ZHANG AND Q. LIU, *1d convolutional neural networks for time series classification*, arXiv preprint, (2023).
- [26] Y. ZHANG, X. LI, AND J. WANG, *A survey on gps spoofing detection and mitigation*, arXiv preprint, (2020).
- [27] Q. ZHAO AND B. LIU, *Px4 autopilot vulnerabilities and security analysis*, arXiv preprint, (2020).

Appendix A

Supplementary Material

A.1 Project Directory Structure

```
gps_spoofing/
|-- gps_spoof.py          # Attack injector
|-- run_tui.py           # TUI management console entry point
|-- start_px4.sh         # PX4 SITL launcher
|-- requirements.txt      # Python dependencies
|-- Step_1_Simulation/   # PX4 SITL + Gazebo
|   +--- PX4-Autopilot/  # Flight stack submodule
|-- Step_2_Monitoring/   # MAVLink telemetry acquisition
|   |-- mavlink_client.py # Connection, threading, message handling
|   |-- state_model.py    # Telemetry dataclasses
|   |-- config.py         # Connection URL, log rate, message types
|   +--- main.py         # CLI entry point
|-- Step_3_Attacks/      # Attack simulation
|   |-- sniff_mavlink.py  # MAVLink traffic sniffer
|   +--- raw_gps_test.py  # Raw GPS injection test
|-- Step_4_Detection/    # ML pipeline
|   |-- pipeline.py       # Pipeline orchestrator
|   |-- live_inference.py # Real-time detector
|   |-- terrain_classifier.py # Flat/mountain/sea classifier
|   |-- terrain_dispatcher.py # Terrain model router
|   |-- demo_dataset.py   # Dataset viewer
|   |-- config/pipeline.yaml # Central configuration
|   +--- scripts/
|       |-- 01_clean_log.py # Data cleaning
|       |-- 02_auto_label.py # Heuristic labelling
|       |-- 03_make_windows.py # Sliding window creation
|       |-- 04_train_baseline.py # RF + CNN training
|       +--- 05_train_terrain_models.py
|-- Step_5_Data/         # Data storage
|   |-- raw/             # Raw CSV logs
|   |-- processed/       # Cleaned + labelled data
|   |-- models/          # Trained models
|   |-- artifacts/       # Dataset, scaler, metadata
|   +--- synthetic/      # Synthetic training data
```

```

|-- Step_6_UI/                                # User interfaces
|   |-- tui_console/                          # Textual-based TUI
|   |   |-- app.py                           # Main dashboard
|   |   |-- config.py                        # Component definitions
|   |   |-- process_manager.py               # PID lifecycle
|   |   +-- widgets.py                       # Component control functions
|   +-- web_ui/                               # Streamlit dashboard
|       |-- app.py                           # Web app
|       +-- log_utils.py                     # Log parsing
|-- Step_7_Research/                          # Academic output (this thesis)
+-- Step_8_Archive/                           # Legacy logs

```

A.2 Configuration File

The central configuration file `Step_4_Detection/config/pipeline.yaml` defines all pipeline parameters:

```

1 data:
2   raw_dir: "Step_5_Data/raw"
3   processed_dir: "Step_5_Data/processed"
4   artifacts_dir: "Step_5_Data/artifacts"
5   models_dir: "Step_5_Data/models"
6
7 cleaning:
8   min_fix_type: 3
9   max_dt_gap_s: 5.0
10  drop_duplicates: true
11
12 windows:
13   length: 30
14   stride: 15
15
16 auto_label:
17   max_normal_speed_ms: 30.0
18   min_satellites: 6
19   max_eph_m: 5.0
20
21 training:
22   model_type: "rf"
23   n_estimators: 100
24
25 inference:
26   anomaly_threshold: 0.3

```

A.3 Commands Reference

Command	Description
<code>python run_tui.py</code>	Start the management console

Command	Description
<code>python -m Step_2_Monitoring.main</code>	Start GPS monitor
<code>python Step_4_Detection/live_inference.py</code>	Start live anomaly detector
<code>streamlit run Step_6_UI/web_ui/app.py</code>	Start web dashboard (port 8501)
<code>python Step_4_Detection/pipeline.py full</code>	Run complete ML pipeline
<code>python gps_spoofers.py --mode drift --intensity 5.0</code>	Launch spoofing attack
<code>./start_px4.sh windy</code>	Start PX4 SITL simulation

A.4 Ethics and Compliance

This research was conducted using simulated environments only. No physical UAV flights or real-world GPS spoofing experiments were performed. All experiments complied with Deakin University's research ethics guidelines. The GPS spoofing attack injector is designed exclusively for testing in controlled simulation environments and should not be used with operational UAV systems.